

Algorithm Problems

05

In This Edition

1	Contents	3
2	Complexity Analysis (Conceptual)	3
	2.1 What is the time complexity of Python's <code>list.sort()</code> ?	3
	2.2 What is the amortized complexity of appending to a Python list?	4
	2.3 Given a recursive function, derive its time complexity using the Master Theorem.	5
	2.4 What is the space complexity of BFS vs DFS?	5
3	Sorting	6
	3.1 Sort an Array of 0s, 1s, and 2s (LC 75)	6
	3.2 Kth Largest Element in an Array (LC 215)	7
	3.3 Check if an Array is Sorted in $O(n)$	8
	3.4 Merge Sorted Array (LC 88)	9
4	Binary Search	10
	4.1 Find First and Last Position of Element in Sorted Array (LC 34)	10
	4.2 Search in Rotated Sorted Array (LC 33)	11
	4.3 Find Minimum in Rotated Sorted Array (LC 153)	12
	4.4 Koko Eating Bananas (LC 875) / Capacity to Ship Packages (LC 1011)	12
	4.5 Find Peak Element (LC 162)	14
5	Graph Algorithms	14
	5.1 Number of Islands (LC 200)	15
	5.2 Course Schedule (LC 207)	16
	5.3 Word Ladder (LC 127)	17
	5.4 Clone Graph (LC 133)	18
	5.5 Cheapest Flights Within K Stops (LC 787)	19
	5.6 Reconstruct Itinerary (LC 332)	19
	5.7 Network Delay Time (LC 743)	21
	5.8 Find if Path Exists in Graph (LC 1971)	22
	5.9 Min Cost to Connect All Points (LC 1584)	23
	5.10 Critical Connections in a Network / Bridges (LC 1192)	24
6	Dynamic Programming	25
	6.1 Climbing Stairs (LC 70)	25
	6.2 Coin Change (LC 322)	26
	6.3 Longest Common Subsequence (LC 1143)	27
	6.4 0/1 Knapsack	27
	6.5 Unbounded Knapsack	28
	6.6 Longest Increasing Subsequence (LC 300)	29
	6.7 Edit Distance (LC 72)	30
	6.8 Regular Expression Matching (LC 10)	31
	6.9 Maximum Subarray --- Kadane's (LC 53)	32
	6.10 Maximum Product Subarray (LC 152)	33

6.14	Minimum Path Sum (LC 64)	35
6.15	Burst Balloons (LC 312)	36
6.16	Word Break (LC 139)	37
6.17	Partition Equal Subset Sum (LC 416)	38
7	Backtracking	39
7.1	N-Queens (LC 51)	39
7.2	Sudoku Solver (LC 37)	40
7.3	Generate Parentheses (LC 22)	41
7.4	Subsets (LC 78)	42
7.5	Permutations (LC 46)	43
7.6	Combinations (LC 77)	44
7.7	Word Search (LC 79)	45
8	Greedy	46
8.1	Merge Intervals (LC 56)	46
8.2	Non-overlapping Intervals (LC 435)	46
8.3	Jump Game (LC 55)	47
8.4	Jump Game II (LC 45)	48
8.5	Task Scheduler (LC 621)	49
8.6	Minimum Number of Arrows to Burst Balloons (LC 452)	50

Companion to **Core Algorithms & Complexity Analysis**. Every "Common Interview Question" listed in that file is solved here in full: the optimal algorithm and why it's the right tool, the explicit recurrence or invariant, clean runnable Python, and the mistakes that sink candidates. Problems are grouped by algorithmic technique.

For the techniques themselves (complexity analysis, the algorithm families, cheat tables), see [03-algorithms-and-complexity.md](#). The data-structure-centric problems (arrays, trees, heaps, graphs as structures, caches) live in [04-data-structure-problems.md](#).

How to read each entry: *Problem* → *Optimal algorithm & complexity* → *State/recurrence or core idea* → *Code* → *Error-prone spots* → *Follow-up*.

1 Contents

- › Complexity Analysis (Conceptual)
- › Sorting
- › Binary Search
- › Graph Algorithms
- › Dynamic Programming
- › Backtracking
- › Greedy

2 Complexity Analysis (Conceptual)

2.1 What is the time complexity of Python's `list.sort()`?

Answer. $O(n \log n)$ in the worst and average case, $O(n)$ in the best case (already-sorted or reverse-sorted input). The algorithm is **Timsort**, an adaptive, stable, hybrid of merge sort and insertion sort. Space is $O(n)$ in the worst case for the temporary merge buffer (the implementation caps it at $n/2$, but that is still $O(n)$).

Why. Timsort exploits pre-existing order in real-world data by working with **runs** — maximal already-sorted contiguous subsequences.

1. **Run detection.** Scan left to right and identify the next run. A run is either ascending ($a[i] \leq a[i+1] \leq \dots$) or strictly descending ($a[i] > a[i+1] > \dots$); a descending run is reversed in place (which is why strict $>$ is used — to keep the sort stable, equal elements must not be reversed).
2. **Minrun extension.** If a natural run is shorter than a computed threshold `minrun` (between 32 and 64, chosen so that n / minrun is close to a power of two), it is extended to `minrun` elements using **binary insertion sort**. Binary insertion sort is $O(k^2)$ for k elements but k is bounded by 64, so this is effectively constant work per run.
3. **Merging.** Runs are pushed onto a stack. Timsort maintains stack invariants (e.g. for adjacent run lengths $x > y + z$ and $y > z$) and merges eagerly to keep the stack balanced. Each element participates in $O(\log n)$ merge levels, giving $O(n \log n)$ merges total. Merging uses **galloping mode**: when one run is consistently winning, it binary-searches ahead to copy a block at once, accelerating merges of skewed-size runs.

Worked best case. If the array is already sorted, run detection finds a single run of length n in one $O(n)$ pass, no merging is needed, and the total cost is $O(n)$. This is the adaptive payoff that plain merge sort (always $O(n \log n)$) does not get.

**** Watch out:****

- › It is **stable** — equal elements keep their original relative order. Do not assume an unstable sort; people sometimes rely on stability for multi-key sorts (sort by secondary key first, then primary).
- › `list.sort()` sorts **in place** and returns `None`; `sorted(iterable)` returns a new list. Writing `x = mylist.sort()` is a classic bug (`x` becomes `None`).
- › "Best case $O(n)$ " requires the data to actually be nearly sorted; random data is still $O(n \log n)$. The constant factor is also higher than a naive quicksort on random data.
- › It is not $O(1)$ space. The merge buffer is $O(n)$.

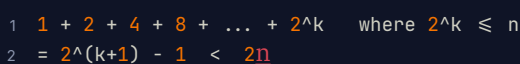
2.2 What is the amortized complexity of appending to a Python list?

Answer.

$O(1)$ amortized. A single append can occasionally cost $O(n)$ (when the backing array is full and must be resized), but the average cost over any sequence of n appends is $O(1)$ per operation, because resizes happen on a geometric schedule.

Why. A CPython `list` is a dynamic array: a contiguous block of pointers with a size (number of used slots) and a capacity (number of allocated slots). When `size == capacity` and you append, the list **over-allocates**: it grows capacity to roughly $1.125 * \text{size} + \text{constant}$ (CPython's growth pattern; the textbook model uses a doubling factor of 2). Growth requires allocating a new buffer and copying all existing elements — an $O(n)$ step. The key is that this expensive step happens rarely, with the gap between resizes growing geometrically.

Aggregate analysis (doubling model, factor 2). Start with capacity 1 and perform n appends. Resizes occur when size crosses 1, 2, 4, 8, ..., up to n . The total copy work is the sum of the sizes at which we resize:



```

1 1 + 2 + 4 + 8 + ... + 2^k   where 2^k ≤ n
2 = 2^(k+1) - 1 < 2n

```

So all resizes together cost less than $2n$ element-copies. Plus n appends each doing $O(1)$ base work. Total = $O(n)$ for n appends, hence $O(1)$ **amortized per append**.

Accounting (banker's) view. Charge 3 units per append: 1 to store the new element, 1 saved to pay for eventually copying this element, and 1 saved to pay for copying an old element that has not yet "saved up" since the last resize. When a resize copies the whole array, every copied element already has a credit to pay for its move. The constant per-op charge (3) proves $O(1)$ amortized.

**** Watch out:****

- › Amortized $O(1)$ is **not** worst-case $O(1)$. A specific append can spike to $O(n)$. For latency-sensitive code (real-time systems), this tail matters.
- › The geometric growth factor must be > 1 (constant multiplicative). If you grew by a fixed **additive** amount (e.g. +10 each time), resizes would be $\Theta(n/10)$ times and total copy work $\Theta(n^2)$ — appends would be $O(n)$ amortized, not $O(1)$.
- › `list.insert(0, x)` (prepend) is $O(n)$ every time — it shifts all elements. Use `collections.deque`

for $O(1)$ front operations.

- › `pop()` from the end is $O(1)$; `pop(0)` is $O(n)$.

2.3 Given a recursive function, derive its time complexity using the Master Theorem.

Answer. For a divide-and-conquer recurrence of the form

```
1 T(n) = a * T(n/b) + f(n), with a ≥ 1, b > 1, f(n) > 0
```

let the **critical exponent** be $c = \log_b(a)$ (the exponent for which n^c matches the total work of the leaves). Compare $f(n)$ against n^c :

- › **Case 1 (leaves dominate).** If $f(n) = O(n^{c - \epsilon})$ for some $\epsilon > 0$, then $T(n) = \Theta(n^c)$.
- › **Case 2 (balanced).** If $f(n) = \Theta(n^c \log^k n)$ for $k \geq 0$, then $T(n) = \Theta(n^c \log^{k+1} n)$. The common subcase $k=0$: $f(n) = \Theta(n^c)$ gives $T(n) = \Theta(n^c \log n)$.
- › **Case 3 (root dominates).** If $f(n) = \Omega(n^{c + \epsilon})$ for some $\epsilon > 0$ **and** the regularity condition $a \cdot f(n/b) \leq k \cdot f(n)$ holds for some $k < 1$ and large n , then $T(n) = \Theta(f(n))$.

Why / intuition. The recursion tree has depth $\log_b(n)$. Level i has a^i subproblems each of size n/b^i , contributing $a^i \cdot f(n/b^i)$ work. The number of leaves is $a^{(\log_b n)} = n^{(\log_b a)} = n^c$. The three cases ask whether the per-level work is geometrically increasing toward the leaves (Case 1, sum dominated by bottom = n^c), roughly equal across all $\log_b n$ levels (Case 2, multiply by $\log n$), or geometrically decreasing (Case 3, sum dominated by the root = $f(n)$).

Worked example 1 – Merge sort. $T(n) = 2 T(n/2) + n$. Here $a=2$, $b=2$, $f(n)=n$. $c = \log_2(2) = 1$, so $n^c = n^1 = n$. Compare $f(n)=n$ with $n^c=n$: they match, so this is **Case 2 with $k=0$** . Result: $T(n) = \Theta(n^c \log n) = \Theta(n \log n)$.

Worked example 2 – Binary search. $T(n) = 1 \cdot T(n/2) + 1$. Here $a=1$, $b=2$, $f(n)=1$ (constant work per call to compute the midpoint and compare). $c = \log_2(1) = 0$, so $n^c = n^0 = 1$. Compare $f(n)=1$ with $n^c=1$: they match ($f = \Theta(1) = \Theta(n^0)$), so **Case 2 with $k=0$** . Result: $T(n) = \Theta(n^c \log n) = \Theta(1 \cdot \log n) = \Theta(\log n)$.

**** Watch out:****

- › The Master Theorem only applies to recurrences of exactly the form $a T(n/b) + f(n)$ with constant a, b . It does **not** apply when subproblem sizes differ (e.g. $T(n) = T(n/3) + T(2n/3) + n$, the worst case of quicksort partitioning – use the recursion-tree method or Akra–Bazzi).
- › There is a **gap** between cases. If $f(n)$ is between polynomially (e.g. $f(n) = n^c / \log n$), none of the three cases applies and you need a generalized version.
- › Case 3 requires the **regularity condition**, not just $f(n) = \Omega(n^{c+\epsilon})$. It almost always holds for polynomial f , but is required for rigor.
- › a is the number of recursive calls, not the depth. Misreading $T(n) = 2T(n/2)$ as $a=1$ is a common error.

2.4 What is the space complexity of BFS vs DFS?

Answer. Both visit every vertex, so both are $O(V)$ time ($O(V+E)$ for a graph). The difference is the **auxiliary space for the frontier**:

- › **BFS** stores a frontier of all nodes at the current "level" in a queue. Space is $O(\text{maximum})$

width of the graph/tree). In the worst case this is $O(V)$ (or $O(b^d)$ for a tree of branching factor b and depth d — the last level can hold up to half of all nodes in a complete binary tree).

- ▶ **DFS** stores the current root-to-node path on the call stack (recursion) or an explicit stack. Space is $O(\text{maximum depth/height})$. In the worst case this is $O(V)$ (a degenerate path graph) — for a balanced tree it is $O(\log n)$; for a skewed tree it is $O(n)$.

Why. BFS explores level by level: before dequeuing level i it must have enqueued all of level $i+1$ that it discovered, so the queue holds an entire level (the width). DFS goes as deep as possible first, keeping only the nodes along the single active path plus their siblings-to-visit, so its memory is bounded by the **height**.

Contrast on tree shapes.

- ▶ **Wide-but-shallow** tree (e.g. a root with 1,000,000 children, depth 2): BFS queue holds $\sim 1,000,000$ nodes $\rightarrow O(\text{width})$ is huge. DFS holds only ~ 2 nodes on the stack $\rightarrow O(\text{depth})$ is tiny. **DFS wins on space.**
- ▶ **Deep-but-narrow** tree (a single chain of 1,000,000 nodes, like a linked list): DFS recursion is 1,000,000 frames deep $\rightarrow O(\text{depth})$ is huge (and may overflow the Python recursion limit, default ~ 1000). BFS queue holds only 1 node at a time $\rightarrow O(\text{width})$ is tiny. **BFS wins on space.**

For a balanced binary tree of n nodes: BFS is $O(n)$ (the bottom level has $\sim n/2$ nodes), DFS is $O(\log n)$ (height). Here DFS is dramatically more space-efficient.

** Watch out:**

- ▶ Both also need a **visited** set on a **graph** (not a tree), which is $O(V)$ regardless — so on general graphs the asymptotic space is $O(V)$ for both; the frontier distinction is what differs in the constant and on trees.
- ▶ Recursive DFS uses the **call stack**; deep graphs can hit Python's recursion limit (`sys.setrecursionlimit`) and crash. Convert to an explicit stack for very deep inputs.
- ▶ BFS is used for **shortest path in unweighted graphs** precisely because it expands by level; DFS does not give shortest paths.
- ▶ "DFS is always cheaper in space" is false — it is only cheaper when the graph is shallower than it is wide.

3 Sorting

3.1 Sort an Array of 0s, 1s, and 2s (LC 75)

Problem. Given an array `nums` containing only the values 0, 1, and 2 (representing red, white, blue), sort it in place so that all 0s come first, then all 1s, then all 2s. Do it in one pass with $O(1)$ extra space, without using a library sort.

Optimal — Dutch National Flag (three pointers) · Time $O(n)$, Space $O(1)$.

Idea. Maintain three regions via pointers `low`, `mid`, `high`: `[0, low)` is all 0s, `[low, mid)` is all 1s, `(high, n-1]` is all 2s, and `[mid, high]` is unprocessed. Scan with `mid`. A 0 is swapped down to the `low` boundary; a 2 is swapped up to the `high` boundary; a 1 is already in place. The crucial asymmetry: after swapping a 2 into position you do **not** advance `mid`, because the element swapped in from the back is unexamined.

```

1 def sortColors(nums):
2     low, mid, high = 0, 0, len(nums) - 1
3     while mid ≤ high:
4         if nums[mid] == 0:
5             nums[low], nums[mid] = nums[mid], nums[low]
6             low += 1
7             mid += 1
8         elif nums[mid] == 1:
9             mid += 1
10        else: # nums[mid] == 2
11            nums[mid], nums[high] = nums[high], nums[mid]
12            high -= 1
13            # do NOT increment mid here
14    return nums

```

**** Error-prone spots:****

- When you swap a 2 to high, **do not advance mid** — the swapped-in value at mid is from the unprocessed tail and must be re-examined. Advancing here is the most common bug.
- When you swap a 0 to low, you **may** advance mid safely, because everything in [low, mid) is 1s, so the element coming from low into mid is a 1 (already processed). This is why the 0-case advances both pointers but the 2-case advances neither.
- Loop condition is $mid \leq high$ (inclusive). Using $mid < high$ skips the final element.
- A self-swap when $low = mid$ is harmless but make sure pointers still advance.

Follow-up. Generalizes to a "3-way partition" used in quicksort to handle many duplicate keys efficiently. For k distinct values, you would use counting sort (two-pass) instead.

3.2 Kth Largest Element in an Array (LC 215)

Problem. Given an integer array `nums` and integer k , return the k -th largest element (k -th in sorted-descending order, allowing duplicates by position, not by distinct value).

Optimal — QuickSelect (Hoare/Lomuto partition) · Time $O(n)$ average, $O(n^2)$ worst, Space $O(1)$ extra (in place).

Idea. The k -th largest is the $(n-k)$ -th smallest in 0-indexed sorted order. QuickSelect partitions the array around a pivot so the pivot lands in its final sorted position p ; then it recurses (or loops) into only the side that contains the target index $n-k$, discarding the other half. Because we only recurse into one side, average work is $n + n/2 + n/4 + \dots = O(n)$. A **randomized pivot** makes the $O(n^2)$ worst case (already-sorted input with bad pivots) vanishingly unlikely.

```

1 import random
2
3 def findKthLargest(nums, k):
4     target = len(nums) - k # k-th largest == target-th smallest (0-indexed)
5     lo, hi = 0, len(nums) - 1
6
7     def partition(lo, hi):
8         # randomized pivot to avoid  $O(n^2)$  on sorted input
9         pivot_idx = random.randint(lo, hi)
10        nums[pivot_idx], nums[hi] = nums[hi], nums[pivot_idx]
11        pivot = nums[hi]

```



```

12     i = lo # boundary: nums[lo..i-1] < pivot
13     for j in range(lo, hi):
14         if nums[j] < pivot:
15             nums[i], nums[j] = nums[j], nums[i]
16             i += 1
17     nums[i], nums[hi] = nums[hi], nums[i] # place pivot at i
18     return i
19
20 while lo ≤ hi:
21     p = partition(lo, hi)
22     if p == target:
23         return nums[p]
24     elif p < target:
25         lo = p + 1
26     else:
27         hi = p - 1
28 return -1 # unreachable for valid input

```

**** Error-prone spots:****

- › The conversion `target = len(nums) - k`. Off-by-one here returns the wrong element; verify with a tiny example ($n=2, k=1 \rightarrow$ target index 1, the larger element).
- › **Always randomize the pivot.** Lomuto with a fixed last-element pivot degrades to $O(n^2)$ on sorted or reverse-sorted input — a classic interview trap that times out on adversarial tests.
- › In Lomuto partition the invariant is `nums[lo..i-1] < pivot`; the final swap places the pivot at index `i`. Forgetting that swap leaves the pivot misplaced.
- › Use an **iterative** narrowing loop (shown) or be careful with recursion depth — recursive QuickSelect on the bad side can hit $O(n)$ stack frames.
- › Comparison direction: this code finds the target-th **smallest**. If you partition for largest instead, the index arithmetic flips.

Follow-up. A heap solution is $O(n \log k)$ using a min-heap of size `k` (`heapq.nlargest`), simpler and with guaranteed worst case — preferable for streaming data. For deterministic $O(n)$ worst case, use **median-of-medians** to pick the pivot (theoretically optimal, large constant).

3.3 Check if an Array is Sorted in $O(n)$

Problem. Given an array, determine whether it is sorted in non-decreasing order. Return True/False in a single linear pass.

Optimal — Linear scan · Time $O(n)$, Space $O(1)$.

Idea. Compare each adjacent pair. The array is sorted iff `nums[i-1] ≤ nums[i]` for every `i`. A single counterexample disproves sortedness, so return False on the first inversion; otherwise True.

```

1 def is_sorted(nums):
2     return all(nums[i] ≤ nums[i + 1] for i in range(len(nums) - 1))

```

Equivalent explicit form (clearer for the boundary):

```

1 def is_sorted_explicit(nums):
2     for i in range(1, len(nums)):
3         if nums[i] < nums[i - 1]:
4             return False
5     return True

```

** Error-prone spots:**

- **Non-decreasing vs strictly increasing.** Use $\leq$ for non-decreasing (allows duplicates). Use $<$ if strict ordering is required — $[1, 1, 2]$ is sorted non-decreasing but not strictly increasing.
- Empty arrays and single-element arrays are trivially sorted; the `range(len(nums)-1) / range(1, len(nums))` bounds handle this correctly (the loop body never runs), but a hand-rolled `range(len(nums))` with `nums[i+1]` would index out of bounds.
- `all()` short-circuits, so it is genuinely $O(n)$ and stops early on the first inversion — no separate early-return needed.
- For descending-sorted checks, flip the comparison; do not forget to pick one direction explicitly.

3.4 Merge Sorted Array (LC 88)

Problem. You are given two sorted integer arrays `nums1` and `nums2`. `nums1` has length $m + n$ where the first m entries are the real elements and the last n are 0 placeholders; `nums2` has n elements. Merge `nums2` into `nums1` so that `nums1` becomes a single sorted array, **in place**, using $O(1)$ extra space.

Optimal — Three pointers filling from the back · Time $O(m + n)$, Space $O(1)$.

Idea. Merging from the front would overwrite unprocessed elements of `nums1`. Instead fill from the **back**: place the largest remaining element at the highest empty slot. Pointer `i` scans `nums1`'s real part from the end, `j` scans `nums2` from the end, and `k` is the write position at the very end of `nums1`. Each step writes the larger of `nums1[i]` / `nums2[j]` into `nums1[k]`. The free space at the tail guarantees we never clobber data we still need.

```

1 def merge(nums1, m, nums2, n):
2     i, j, k = m - 1, n - 1, m + n - 1
3     while j >= 0:                                # only need to exhaust nums2
4         if i >= 0 and nums1[i] > nums2[j]:
5             nums1[k] = nums1[i]
6             i -= 1
7         else:
8             nums1[k] = nums2[j]
9             j -= 1
10        k -= 1

```

** Error-prone spots:**

- Loop on `while j >= 0`, not `while i >= 0 and j >= 0`. If `nums1`'s elements run out first ($i < 0$), the remaining `nums2` values must still be copied; the `if i >= 0 and ...` guard handles this by falling through to copy `nums2[j]`.
- If `nums2` empties first ($j < 0$), you can stop immediately — the remaining `nums1` elements are already in their correct positions (no copying needed). That is exactly why the loop condition is `j >= 0`.

- › Initialize $k = m + n - 1$ (last index), $i = m - 1$, $j = n - 1$. Off-by-one on any of these corrupts the merge.
- › Use strict $>$ (write from `nums1` when it is strictly larger) so ties pull from `nums2` — either tie-break works for correctness here, but be consistent.
- › This relies on the tail of `nums1` being available scratch space; it does not work if `nums1` has no slack.

4 Binary Search

4.1 Find First and Last Position of Element in Sorted Array (LC 34)

Problem. Given a sorted array `nums` and a target value, return `[first_index, last_index]` of target. If absent, return `[-1, -1]`. Must run in $O(\log n)$.

Optimal — Two boundary binary searches (leftmost + rightmost) · Time $O(\log n)$, Space $O(1)$.

Idea. Run two separate binary searches: one finds the **leftmost** index where target could be inserted (lower bound), the other finds the leftmost index where `target + 1` could be inserted — that minus one is the **rightmost** occurrence. Using `bisect_left` semantics on both gives clean, off-by-one-safe boundaries.

```

1 def searchRange(nums, target):
2     def lower_bound(x):
3         # leftmost index i such that nums[i] ≥ x; returns len(nums) if none
4         lo, hi = 0, len(nums)          # half-open [lo, hi)
5         while lo < hi:
6             mid = (lo + hi) // 2
7             if nums[mid] < x:
8                 lo = mid + 1
9             else:
10                hi = mid
11        return lo
12
13    left = lower_bound(target)
14    if left == len(nums) or nums[left] != target:
15        return [-1, -1]
16    right = lower_bound(target + 1) - 1
17    return [left, right]

```

**** Error-prone spots:****

- › Use a **half-open** interval `[lo, hi)` with `hi = len(nums)` and loop `while lo < hi`, updating `hi = mid` (not `mid - 1`) on the $\geq$ branch. Mixing closed-interval updates (`hi = mid - 1`) with this template causes infinite loops or missed boundaries.
- › After `lower_bound(target)`, you **must** check both `left < len(nums)` and `nums[left] == target`; otherwise you can return a position where the target was merely “insertable” but absent.
- › `Rightmost = lower_bound(target + 1) - 1`. Computing it as `upper_bound` is equivalent; the `-1` is essential.
- › `mid = (lo + hi) // 2` biases left, which is required for the `hi = mid` update to make progress. A right-biased `mid` with `hi = mid` loops forever.
- › Equivalent with the standard library: `bisect.bisect_left(nums, target)` and `bisect.bisect_right(nums,`

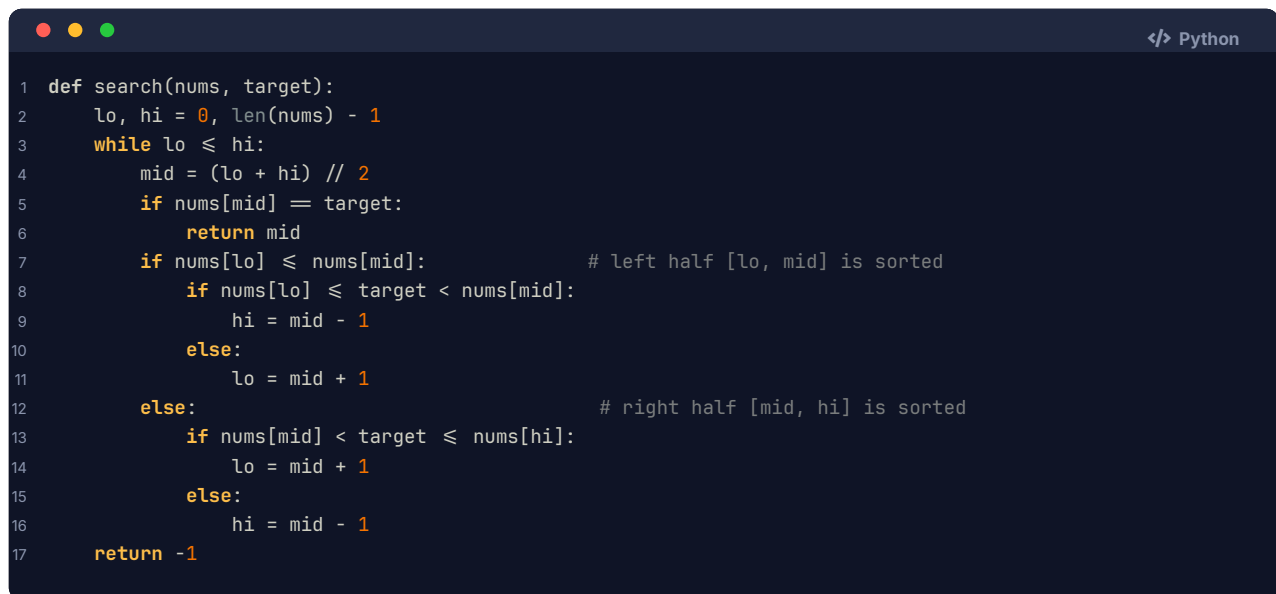
target) - 1.

4.2 Search in Rotated Sorted Array (LC 33)

Problem. A sorted ascending array of distinct values was rotated at an unknown pivot (e.g. [4,5,6,7,0,1,2]). Given the array and a target, return its index or -1, in $O(\log n)$.

Optimal – Modified binary search (identify the sorted half) · Time $O(\log n)$, Space $O(1)$.

Idea. At any midpoint, **at least one half** ([lo, mid] or [mid, hi]) is fully sorted, detectable by comparing endpoints. Determine which half is sorted, check whether target lies within that sorted half's value range; if so search there, otherwise search the other half. This halves the search space each step despite the rotation.



```

1 def search(nums, target):
2     lo, hi = 0, len(nums) - 1
3     while lo ≤ hi:
4         mid = (lo + hi) // 2
5         if nums[mid] == target:
6             return mid
7         if nums[lo] ≤ nums[mid]:          # left half [lo, mid] is sorted
8             if nums[lo] ≤ target < nums[mid]:
9                 hi = mid - 1
10            else:
11                lo = mid + 1
12        else:                             # right half [mid, hi] is sorted
13            if nums[mid] < target ≤ nums[hi]:
14                lo = mid + 1
15            else:
16                hi = mid - 1
17    return -1

```

**** Error-prone spots:****

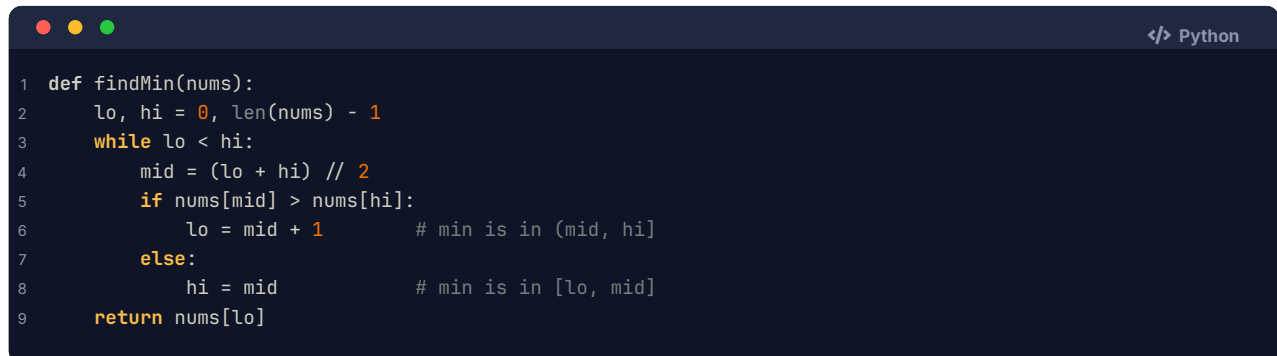
- The sortedness test must be `nums[lo] ≤ nums[mid]` (with `≤`, not `<`) to correctly handle the case `lo == mid` (two-element or one-element window). Using `<` misclassifies the sorted half and can miss the target.
- The range checks are **half-open on the mid side**: `nums[lo] ≤ target < nums[mid]` and `nums[mid] < target ≤ nums[hi]`. The strict bound at `mid` is correct because `nums[mid] ≠ target` was already ruled out; mixing up which side gets `<` vs `≤` is the most common bug.
- Use closed interval `[lo, hi]` with `lo ≤ hi` and updates `mid - 1` / `mid + 1`. Be consistent – do not blend with the half-open template.
- This version assumes **distinct** values. With duplicates (LC 81), `nums[lo] == nums[mid] == nums[hi]` is ambiguous and you must shrink `lo += 1`; `hi -= 1`, degrading the worst case to $O(n)$.

4.3 Find Minimum in Rotated Sorted Array (LC 153)

Problem. Given a rotated sorted array of **distinct** values, return the minimum element in $O(\log n)$.

Optimal — Binary search comparing mid to the right endpoint · Time $O(\log n)$, Space $O(1)$.

Idea. The minimum is the unique "rotation point" — the only element smaller than its predecessor. Compare `nums[mid]` to `nums[hi]` (the rightmost element of the current window). If `nums[mid] > nums[hi]`, the rotation point (minimum) is strictly to the right of `mid`, so move `lo = mid + 1`. Otherwise the minimum is at `mid` or to its left, so `hi = mid`. The loop converges to the minimum.



```

1 def findMin(nums):
2     lo, hi = 0, len(nums) - 1
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if nums[mid] > nums[hi]:
6             lo = mid + 1      # min is in (mid, hi]
7         else:
8             hi = mid          # min is in [lo, mid]
9     return nums[lo]

```

**** Error-prone spots:****

- ▶ Compare to `nums[hi]`, **not** `nums[lo]`. Comparing to the left endpoint fails on a non-rotated (already sorted) array because `nums[mid] ≥ nums[lo]` always holds there, giving no information.
- ▶ Loop condition is `while lo < hi` (strict), and on the "min is here or left" branch use `hi = mid` (not `mid - 1`) — discarding `mid` could throw away the actual minimum.
- ▶ On the other branch use `lo = mid + 1` — `nums[mid] > nums[hi]` guarantees `mid` is not the minimum, so excluding it is safe and necessary for progress.
- ▶ Terminate by returning `nums[lo]` (`= nums[hi]` at convergence). Do not return `nums[mid]` from inside the loop.
- ▶ Assumes distinct values; with duplicates (LC 154) the `nums[mid] == nums[hi]` tie forces `hi -= 1`, making the worst case $O(n)$.

4.4 Koko Eating Bananas (LC 875) / Capacity to Ship Packages (LC 1011)

Problem. Koko has piles of bananas and h hours. At an eating speed k bananas/hour she needs $\text{ceil}(\text{pile} / k)$ hours per pile. Find the **minimum integer speed** k that lets her finish all piles within h hours. (LC 1011 is the same shape: minimum daily ship capacity to deliver all weights within d days, with the added constraint that capacity must be at least the heaviest single package.)

Optimal — Binary search on the answer space with a monotone feasibility check · Time $O(n \log M)$, Space $O(1)$. (n = number of piles, M = max pile size / answer range.)

Idea. The answer is monotone: if speed k works, every speed $> k$ also works; if k is too slow, every speed $< k$ is too slow. This boolean "feasible(k)" function is a step function (False...False True...True), so binary-search the **smallest k that is feasible**. Search space is $[1, \max(\text{piles})]$ for Koko; the feasibility check sums $\text{ceil}(\text{pile} / k)$ hours.

```

1 import math
2
3 def minEatingSpeed(piles, h):
4     def hours_needed(k):
5         return sum(math.ceil(p / k) for p in piles)
6
7     lo, hi = 1, max(piles)          # k=1 slowest, k=max(piles) finishes 1 pile/hour
8     while lo < hi:
9         mid = (lo + hi) // 2
10        if hours_needed(mid) ≤ h:    # feasible → try slower (smaller k)
11            hi = mid
12        else:                       # too slow → need faster
13            lo = mid + 1
14    return lo

```

For LC 1011 (ship within D days), the answer range lower bound is $\max(\text{weights})$ (one item per day must still fit) and upper bound is $\text{sum}(\text{weights})$ (ship everything in one day):

```

1 def shipWithinDays(weights, D):
2     def days_needed(cap):
3         days, cur = 1, 0
4         for w in weights:
5             if cur + w > cap:
6                 days += 1
7                 cur = 0
8             cur += w
9         return days
10
11    lo, hi = max(weights), sum(weights)
12    while lo < hi:
13        mid = (lo + hi) // 2
14        if days_needed(mid) ≤ D:
15            hi = mid
16        else:
17            lo = mid + 1
18    return lo

```

**** Error-prone spots:****

- **Search the answer, not the array.** The array is not what you binary-search; you binary-search the candidate speed/capacity and use a feasibility predicate.
- Get the bounds right. Koko: $lo = 1$ (speed 0 is invalid — division by zero), $hi = \max(\text{piles})$ (faster never needed since you can only eat one pile per hour anyway). Ship: $lo = \max(\text{weights})$ (must hold the heaviest item), $hi = \text{sum}(\text{weights})$.
- The “find minimum feasible” template uses $while\ lo < hi$, $hi = mid$ on feasible, $lo = mid + 1$ on infeasible, $return\ lo$. This converges to the smallest feasible value. Using $hi = mid - 1$ here can skip the answer.
- Use $\text{math.ceil}(p / k)$ for hours — integer floor division $p // k$ undercounts when there is a remainder. Equivalent integer-only form: $(p + k - 1) // k$ (avoids float rounding issues on huge inputs).
- Verify the predicate direction: smaller $k \rightarrow$ more hours (harder). Inverting the $\leq h$ comparison searches the wrong direction.

4.5 Find Peak Element (LC 162)

Problem. A peak element is one strictly greater than its neighbors. Given an array `nums` where `nums[i] ≠ nums[i+1]` for all valid `i`, and with conceptual `nums[-1] = nums[n] = -infinity`, return the index of **any** peak. Must run in $O(\log n)$ even though the array is **not sorted**.

Optimal — Binary search on the slope direction · Time $O(\log n)$, Space $O(1)$.

Idea. Even without sortedness, you can halve the search space by following the **uphill** direction. At `mid`, compare `nums[mid]` to `nums[mid+1]`. If `nums[mid] < nums[mid+1]`, the slope rises to the right, so a peak must exist somewhere in `[mid+1, hi]` (the array eventually falls to `-infinity` at the boundary). Otherwise the slope falls (or `mid` is itself a peak), so a peak exists in `[lo, mid]`. Because the boundaries act as `-infinity`, an ascending segment is guaranteed to crest into a peak.

```

1 def findPeakElement(nums):
2     lo, hi = 0, len(nums) - 1
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if nums[mid] < nums[mid + 1]:
6             lo = mid + 1      # peak is to the right
7         else:
8             hi = mid          # peak is at mid or to the left
9     return lo                # lo = hi: a peak

```

Why this terminates at a peak. The invariant is that the window `[lo, hi]` always contains a peak (treating off-array neighbors as `-infinity`). When `nums[mid] < nums[mid+1]` we discard `[lo, mid]` keeping the rising side; when `nums[mid] > nums[mid+1]` we keep `[lo, mid]` because `mid` is at least a local high relative to its right neighbor. The window shrinks until `lo = hi`, which must be a peak.

**** Error-prone spots:****

- ▶ Compare `nums[mid]` to `nums[mid + 1]`, and use `while lo < hi` so that `mid + 1 ≤ hi` is always in range — this avoids an out-of-bounds access. With `lo ≤ hi` you could compute `mid = hi` and read `nums[hi+1]`.
- ▶ On the rising branch use `lo = mid + 1` (exclude `mid`); on the other branch use `hi = mid` (include `mid`, since `mid` may be the peak). Swapping these breaks the invariant.
- ▶ It returns **any** peak, not the global maximum. Do not assume the result is the largest element.
- ▶ The strict-inequality / distinct-adjacent guarantee (`nums[i] ≠ nums[i+1]`) is what makes the comparison unambiguous; with equal neighbors the slope test is undefined and the method may fail.
- ▶ Do not mistake this for needing a sorted array — the whole point is that $O(\log n)$ works on unsorted data via local slope information.

5 Graph Algorithms

5.1 Number of Islands (LC 200)

Problem. Given an $m \times n$ grid of '1' (land) and '0' (water), count the number of islands. An island is a maximal group of land cells connected 4-directionally (up/down/left/right).

Algorithm — DFS/BFS flood fill · Time $O(V + E) = O(mn)$, Space $O(mn)$ worst case. Each cell is a node; edges connect orthogonal land neighbors. Counting islands is counting connected components, and a single traversal visits every cell at most once. Flood fill from each unvisited land cell is the canonical connected-components technique on a grid.

Idea. Scan the grid; whenever you hit an unvisited '1', increment the island count and flood-fill that entire component, marking visited cells so they are never re-counted. The flood fill (DFS via stack/recursion or BFS via queue) consumes the whole island before the outer scan resumes. Sinking visited land in place ('1' $\rightarrow$ '0') avoids a separate visited set.

```

1 def numIslands(grid: list[list[str]]) → int:
2     if not grid:
3         return 0
4     rows, cols = len(grid), len(grid[0])
5     count = 0
6
7     def dfs(r: int, c: int) → None:
8         # iterative flood fill to avoid recursion-depth blowups
9         stack = [(r, c)]
10        grid[r][c] = '0' # sink on push
11        while stack:
12            i, j = stack.pop()
13            for di, dj in ((1, 0), (-1, 0), (0, 1), (0, -1)):
14                ni, nj = i + di, j + dj
15                if 0 ≤ ni < rows and 0 ≤ nj < cols and grid[ni][nj] == '1':
16                    grid[ni][nj] = '0' # sink immediately when discovered
17                    stack.append((ni, nj))
18
19        for r in range(rows):
20            for c in range(cols):
21                if grid[r][c] == '1':
22                    count += 1
23                    dfs(r, c)
24        return count

```

**** Error-prone spots:****

- › Marking a cell visited only when *popped* (not when *pushed*) lets it enter the stack multiple times, risking double counting and blowup; sink on discovery.
- › Recursive DFS can hit Python's recursion limit (~1000) on a large single island; prefer the iterative stack or BFS queue.
- › Out-of-bounds checks must precede the `grid[ni][nj]` read, or you index-error on borders.
- › Only 4-directional moves count here; adding diagonals silently changes the answer.
- › Mutating the input grid is fine for counting but destroys it; clone or use a visited set if the caller needs the grid intact.

Follow-up. Number of Distinct Islands (LC 694) compares island *shapes* by normalizing each flood-fill path; Max Area of Island (LC 695) returns the largest component size.

5.2 Course Schedule (LC 207)

Problem. Given `numCourses` and a list of prerequisite pairs `[a, b]` meaning "to take `a` you must first take `b`", determine whether you can finish all courses (i.e., the prerequisite graph has no cycle).

Algorithm — Topological sort via Kahn's algorithm (BFS) · Time $O(V + E)$, Space $O(V + E)$.

A valid course ordering exists iff the directed graph is a DAG. Kahn's algorithm repeatedly removes in-degree-0 nodes; it processes exactly the nodes reachable in topological order, so it both produces an ordering and detects a cycle (any node left unprocessed lies on a cycle).

Idea. Build edges $b \rightarrow a$ (prerequisite points to dependent) and compute in-degrees. Seed a queue with all in-degree-0 courses, then repeatedly pop a course, "complete" it, and decrement neighbors' in-degrees, enqueueing any that drop to 0. If the number of completed courses equals `numCourses`, no cycle exists; otherwise a cycle blocked some courses from ever reaching in-degree 0.

```

1 from collections import deque
2
3 def canFinish(numCourses: int, prerequisites: list[list[int]]) → bool:
4     adj = [[] for _ in range(numCourses)]
5     indegree = [0] * numCourses
6     for course, prereq in prerequisites:
7         adj[prereq].append(course) # prereq must come before course
8         indegree[course] += 1
9
10    queue = deque(c for c in range(numCourses) if indegree[c] == 0)
11    completed = 0
12    while queue:
13        node = queue.popleft()
14        completed += 1
15        for nxt in adj[node]:
16            indegree[nxt] -= 1
17            if indegree[nxt] == 0:
18                queue.append(nxt)
19    return completed == numCourses

```

**** Error-prone spots:****

- Edge direction: `[a, b]` means $b \rightarrow a$. Reversing it inverts the topological order and breaks in-degree counting.
- Seed the queue with *all* in-degree-0 nodes, not just node 0; disconnected components and multiple roots are common.
- Compare `completed == numCourses`, not "queue emptied" — the queue always empties; the count is what reveals a cycle.
- Duplicate prerequisite edges inflate in-degree; if duplicates are possible and must be ignored, dedupe with sets.
- DFS three-color (white/gray/black) detection is an alternative, but mark a node "fully done" (black) only after recursing all children — marking on entry misses cycles.

Follow-up. Course Schedule II (LC 210) returns the actual ordering (the Kahn pop order); an empty list signals a cycle.

5.3 Word Ladder (LC 127)

Problem. Given `beginWord`, `endWord`, and a `wordList`, transform `beginWord` into `endWord` changing one letter at a time, where every intermediate word must be in `wordList`. Return the number of words in the shortest transformation sequence (including both ends), or 0 if impossible.

Algorithm — BFS on an implicit graph · Time $O(N * L^2)$ for N words of length L (with the wildcard-pattern index), Space $O(N * L^2)$. Words are nodes; an edge joins two words differing in exactly one letter. BFS explores level by level, so the first time `endWord` is dequeued, its level equals the shortest path length — the defining property of BFS for unweighted shortest paths.

Idea. Building the full adjacency by comparing every pair is $O(N^2 * L)$; instead, precompute a pattern index mapping each "wildcard" form like `h*t` to the words that match it, giving $O(N * L)$ generic-state nodes and $O(L)$ neighbor generation per word. Run BFS from `beginWord`, tracking the level (number of words so far). Mark patterns/words visited to avoid revisiting, and return the level upon reaching `endWord`.

```

1  from collections import defaultdict, deque
2
3  def ladderLength(beginWord: str, endWord: str, wordList: list[str]) → int:
4      word_set = set(wordList)
5      if endWord not in word_set:
6          return 0
7
8      L = len(beginWord)
9      patterns = defaultdict(list)
10     for word in word_set | {beginWord}:
11         for i in range(L):
12             patterns[word[:i] + '*' + word[i+1:]].append(word)
13
14     queue = deque([(beginWord, 1)])
15     visited = {beginWord}
16     while queue:
17         word, level = queue.popleft()
18         if word == endWord:
19             return level
20         for i in range(L):
21             pat = word[:i] + '*' + word[i+1:]
22             for nbr in patterns[pat]:
23                 if nbr not in visited:
24                     visited.add(nbr)
25                     queue.append((nbr, level + 1))
26     return 0

```

**** Error-prone spots:****

- Early-exit: return 0 immediately if `endWord` is absent from the list — otherwise BFS runs fruitlessly.
- The count includes both endpoints; start the level at 1 (for `beginWord`), not 0.
- `beginWord` may not be in `wordList` but must seed BFS and contribute to the pattern index.
- Mark a word visited *when enqueued*, not when dequeued, to prevent the same word entering the queue many times.
- Naive all-pairs adjacency is $O(N^2 * L)$ and TLEs on large inputs; the wildcard-pattern bucket is the standard fix.

Follow-up. Word Ladder II (LC 126) returns *all* shortest sequences — run BFS to record parents level by level, then DFS-backtrack the parent DAG. Bidirectional BFS (search from both ends, meet in the middle) roughly square-roots the explored frontier.

5.4 Clone Graph (LC 133)

Problem. Given a reference to a node in a connected undirected graph (each node has an `int val` and a list of `neighbors`), return a deep copy: new nodes mirroring the structure, sharing no objects with the original.

Algorithm — DFS/BFS traversal with a visited/clone map · Time $O(V + E)$, Space $O(V)$.

Cloning is a single graph traversal; the only subtlety is cycles. A hash map `original → clone` doubles as the visited set and the lookup table, ensuring each node is copied exactly once and that back-edges in cycles reconnect to the already-created clone instead of recursing forever.

Idea. When first visiting a node, create its clone and record it in the map *before* recursing into neighbors — this breaks cycles. For each neighbor, either reuse its existing clone from the map or recursively clone it, then append to the current clone's neighbor list. Returning the clone of the entry node yields the full copy.

```

1 class Node:
2     def __init__(self, val=0, neighbors=None):
3         self.val = val
4         self.neighbors = neighbors if neighbors is not None else []
5
6 def cloneGraph(node: 'Node') → 'Node':
7     if node is None:
8         return None
9     clones = {} # original → clone
10
11     def dfs(orig: 'Node') → 'Node':
12         if orig in clones:
13             return clones[orig]
14         copy = Node(orig.val)
15         clones[orig] = copy # register BEFORE recursing (cycle-safe)
16         for nbr in orig.neighbors:
17             copy.neighbors.append(dfs(nbr))
18         return copy
19
20     return dfs(node)

```

**** Error-prone spots:****

- Register the clone in the map *before* recursing into neighbors; doing it after causes infinite recursion on any cycle.
- Handle the `node is None` (empty graph) input explicitly.
- The map key must be the original node object (or its unique `val`), not the clone, or lookups never hit.
- Append clones of neighbors, never the originals — a single leaked original pointer fails the deep-copy check.
- For very large graphs, recursive DFS may overflow the stack; an iterative BFS with a queue and the same map is equivalent.

Follow-up. Copy List with Random Pointer (LC 138) is the same map-based clone over a linked structure with arbitrary pointers.

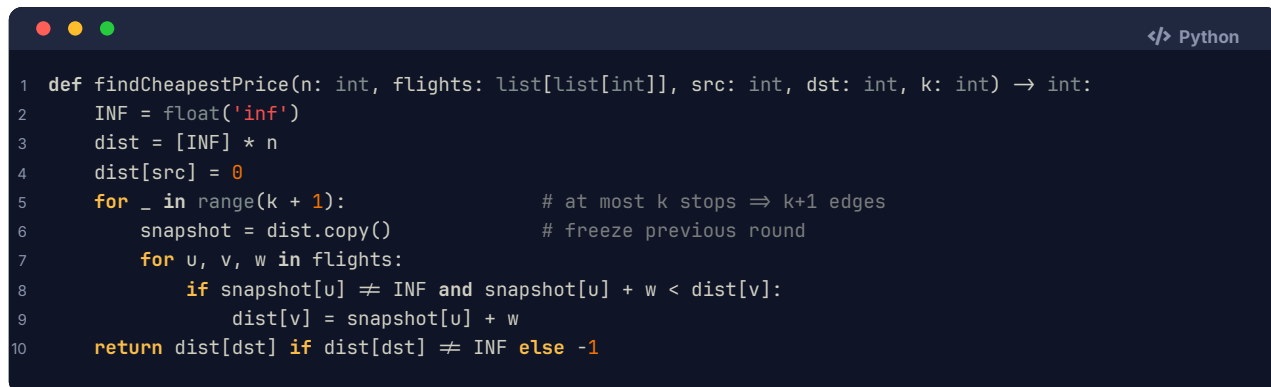
5.5 Cheapest Flights Within K Stops (LC 787)

Problem. Given n cities, a list of directed flights $[u, v, w]$ (price w), source src , destination dst , and k , return the cheapest price from src to dst using at most k stops (i.e., at most $k+1$ edges), or -1 if unreachable within that bound.

Algorithm — Bellman-Ford bounded to $k+1$ relaxation rounds · Time $O(k * E)$, Space $O(n)$.

The "at most k stops" constraint adds a *hop budget* that plain Dijkstra cannot honor: Dijkstra finalizes a node by minimum *cost* and discards costlier paths, but a costlier path using fewer hops may be the only one that can still reach dst within budget. Bellman-Ford naturally tracks paths by edge count — running exactly $k+1$ relaxation rounds yields the cheapest cost using at most $k+1$ edges.

Idea. Maintain $dist[]$ = cheapest cost to each city using at most the rounds processed so far. Each round relaxes every edge, but to enforce the hop bound you must relax from a *snapshot* of the previous round's distances — otherwise edges chained within the same round let a single round span multiple hops. After $k+1$ rounds, $dist[dst]$ is the answer.



```

1 def findCheapestPrice(n: int, flights: list[list[int]], src: int, dst: int, k: int) -> int:
2     INF = float('inf')
3     dist = [INF] * n
4     dist[src] = 0
5     for _ in range(k + 1):
6         snapshot = dist.copy()
7         for u, v, w in flights:
8             if snapshot[u] != INF and snapshot[u] + w < dist[v]:
9                 dist[v] = snapshot[u] + w
10    return dist[dst] if dist[dst] != INF else -1

```

**** Error-prone spots:****

- › Relax from a *snapshot* of the prior round, not the live $dist$; updating in place lets one round traverse several edges, exceeding the hop limit and undercounting cost.
- › Run exactly $k + 1$ rounds (k stops means $k+1$ flights), an off-by-one trap.
- › Skip relaxing from unreachable nodes ($snapshot[u] = INF$) to avoid $inf + w$ arithmetic noise and incorrect updates.
- › Plain Dijkstra by cost is wrong here; if you adapt Dijkstra, the state must be $(cost, node, stops)$ and you must *not* mark a node permanently finalized.
- › Return -1 , not inf , when dst stays unreachable.

Follow-up. A Dijkstra-style variant with a priority queue keyed $(cost, node, stops_remaining)$ also works and can be faster when dst is reached early, but it must allow re-expanding a node at a lower stop count.

5.6 Reconstruct Itinerary (LC 332)

Problem. Given a list of airline tickets $[from, to]$, reconstruct the itinerary in order. All tickets belong to a man who starts at "JFK", every ticket is used exactly once, and a valid itinerary always exists. If multiple valid itineraries exist, return the one that is smallest in lexical order when read as a single string.

Algorithm — Eulerian path via Hierholzer's algorithm · Time $O(E \log E)$, Space $O(E)$. Using every ticket exactly once is exactly an Eulerian path (traverse every edge once) in the directed multigraph of airports. Hierholzer's algorithm constructs such a path in linear time over edges;

sorting each airport's destinations and always taking the lexicographically smallest available edge first yields the smallest itinerary. The $\log E$ factor is the sorting of adjacency lists.

Idea. Sort each node's outgoing destinations so we greedily consume the smallest first. Run an iterative DFS: push the current airport, and whenever it still has unused outgoing edges, descend into its smallest destination; when an airport has no more edges, it is a "dead end" — pop it onto the result. The result built in pop order is the Eulerian path *reversed*, so reverse it at the end. This "post-order" emission correctly handles the dead-end vertex without getting stuck.



```

1 from collections import defaultdict
2
3 def findItinerary(tickets: list[list[str]]) → list[str]:
4     graph = defaultdict(list)
5     for src, dst in sorted(tickets, reverse=True):
6         graph[src].append(dst) # reverse-sorted so pop() yields smallest dst
7
8     route = []
9     stack = ['JFK']
10    while stack:
11        node = stack[-1]
12        if graph[node]:
13            stack.append(graph[node].pop()) # take smallest unused edge
14        else:
15            route.append(stack.pop())      # dead end: emit in post-order
16    return route[::-1]

```

**** Error-prone spots:****

- › Emit a node only when it has *no* remaining edges (post-order), then reverse — emitting in visit order fails when the path must backtrack through the start.
- › Sorting tickets in reverse lets `list.pop()` (from the end) return the lexicographically smallest destination cheaply.
- › It is a *multigraph*: the same [from, to] ticket can appear several times; never deduplicate edges.
- › Plain greedy DFS that just follows the smallest edge can dead-end early and strand unused tickets; Hierholzer's post-order is what makes it correct.
- › Start the stack at "JFK" regardless of lexical order of airports.

Follow-up. The same Hierholzer technique solves Eulerian circuits; checking *existence* of an Eulerian path requires the degree conditions (at most one vertex with out-in = 1, one with in-out = 1, rest balanced) plus connectivity.

5.7 Network Delay Time (LC 743)

Problem. A network of n nodes; $\text{times}[i] = [u, v, w]$ is a directed edge with travel time w . Send a signal from node k ; return the minimum time for *all* nodes to receive it, or -1 if some node is unreachable. The answer is the maximum shortest-path distance from k .

Algorithm — Dijkstra with a binary min-heap · Time $O(E \log V)$, Space $O(V + E)$. Single-source shortest paths with non-negative weights is the textbook case for Dijkstra. The min-heap always extracts the closest unfinalized node, so each node is finalized with its true shortest distance; the answer is the largest of those distances. Non-negative weights are what make the greedy “finalize-on-pop” valid.

Idea. Maintain $\text{dist}[]$ initialized to infinity except $\text{dist}[k] = 0$. Repeatedly pop the smallest (d, node) from the heap; if d exceeds the recorded $\text{dist}[\text{node}]$, it's a stale entry — skip it. Otherwise relax each outgoing edge and push improved neighbors. After the heap drains, if every node was reached, the answer is $\max(\text{dist})$; otherwise -1 .

```

1 import heapq
2 from collections import defaultdict
3
4 def networkDelayTime(times: list[list[int]], n: int, k: int) → int:
5     graph = defaultdict(list)
6     for u, v, w in times:
7         graph[u].append((v, w))
8
9     dist = {}                                # node → shortest distance (finalized on pop)
10    heap = [(0, k)]                           # (distance, node)
11    while heap:
12        d, node = heapq.heappop(heap)
13        if node in dist:                       # already finalized ⇒ stale entry
14            continue
15        dist[node] = d
16        for nbr, w in graph[node]:
17            if nbr not in dist:
18                heapq.heappush(heap, (d + w, nbr))
19    return max(dist.values()) if len(dist) == n else -1

```

**** Error-prone spots:****

- › Skip stale heap entries: a node can be pushed several times with different tentative distances; the first pop is the smallest, so ignore later pops (check `node in dist`).
- › Finalize a node only on pop, never on push — pushing the same node multiple times is expected and correct.
- › The answer is $\max(\text{dist})$ over *all* nodes, not the sum or the distance to a single target.
- › Verify reachability via `len(dist) == n` before returning; an unreachable node makes the answer -1 .
- › Nodes are typically 1-indexed ($1..n$); a 0-indexed `dist` array off-by-one is a classic slip.

Follow-up. With a Fibonacci heap, Dijkstra is $O(E + V \log V)$; for dense graphs an $O(V^2)$ array-scan Dijkstra can beat the heap. If negative edges existed, switch to Bellman-Ford or SPFA.

5.8 Find if Path Exists in Graph (LC 1971)

Problem. Given an undirected graph with n vertices and an edge list `edges`, and two vertices `source` and `destination`, return `True` if a path connects `source` to `destination`.

Algorithm — Union-Find (disjoint set union) with path compression + union by rank · Time $O(E * a(n))$ build, near- $O(1)$ query, Space $O(n)$. The question is pure connectivity: are `source` and `destination` in the same connected component? DSU answers this in almost-constant amortized time per operation (inverse Ackermann $a(n)$), beating a fresh BFS/DFS especially when many connectivity queries are made on a static graph.

Idea. Union every edge's endpoints to merge their components, then test whether `find(source) == find(destination)`. Path compression flattens trees during `find`; union by rank attaches the shorter tree under the taller to keep depth tiny. A plain BFS/DFS reachability check is equally correct for a single query but does not give the reusable component structure.

```

1 class UnionFind:
2     def __init__(self, n: int):
3         self.parent = list(range(n))
4         self.rank = [0] * n
5
6     def find(self, x: int) → int:
7         while self.parent[x] ≠ x:
8             self.parent[x] = self.parent[self.parent[x]] # path halving
9             x = self.parent[x]
10        return x
11
12    def union(self, a: int, b: int) → bool:
13        ra, rb = self.find(a), self.find(b)
14        if ra == rb:
15            return False # already connected
16        if self.rank[ra] < self.rank[rb]:
17            ra, rb = rb, ra
18        self.parent[rb] = ra
19        if self.rank[ra] == self.rank[rb]:
20            self.rank[ra] += 1
21        return True
22
23    def validPath(n: int, edges: list[list[int]], source: int, destination: int) → bool:
24        if source == destination:
25            return True
26        uf = UnionFind(n)
27        for a, b in edges:
28            uf.union(a, b)
29        return uf.find(source) == uf.find(destination)

```

**** Error-prone spots:****

- Handle `source == destination` up front; otherwise a self-query with no edges still works here but the early return is cleaner and safe.
- Compare *roots* (`find(source) == find(destination)`), never raw labels — two nodes may share a root without equal parent entries before compression.
- Union by rank vs. union by size: don't mix the two metrics in one structure or the depth bound degrades.
- Path compression mutates `parent` during `find`; ensure `find` is the only place ranks change and that you compress on every call.
- For directed graphs DSU tests weak connectivity only; for directed reachability use BFS/DFS

instead.

Follow-up. DSU also underlies Kruskal's MST, Number of Connected Components (LC 323), Redundant Connection (LC 684, the first union that returns False is the cycle edge), and Accounts Merge (LC 721).

5.9 Min Cost to Connect All Points (LC 1584)

Problem. Given points on a 2D plane, the cost to connect two points is their Manhattan distance $|x_1 - x_2| + |y_1 - y_2|$. Return the minimum total cost to connect *all* points so that there is exactly one path between any two (a spanning tree).

Algorithm — Prim's MST with a min-heap · Time $O(E \log V) = O(n^2 \log n)$, Space $O(n^2)$ (or $O(n)$ with the eager array variant). The graph is *complete* (every pair connectable), so it has n^2 implicit edges. Prim's grows the MST from a seed vertex, repeatedly adding the cheapest edge crossing the cut between in-tree and out-of-tree vertices — ideal for dense graphs since it never needs to materialize all edges at once.

Idea. Start from point 0; maintain a min-heap of (cost, point) candidates to attach uncovered points. Pop the cheapest edge to an unvisited point, add its cost, mark it in-tree, then push edges from the newly added point to all remaining points. Stop once all n points are in the tree. Skipping already-visited pops handles the duplicate-edge entries.



```

1  import heapq
2
3  def minCostConnectPoints(points: List[List[int]]) -> int:
4      n = len(points)
5      if n <= 1:
6          return 0
7
8      in_mst = [False] * n
9      heap = [(0, 0)]      # (edge_cost, point_index), seed point 0 with cost 0
10     total = 0
11     count = 0
12     while heap and count < n:
13         cost, u = heapq.heappop(heap)
14         if in_mst[u]:
15             continue      # stale entry for an already-attached point
16         in_mst[u] = True
17         total += cost
18         count += 1
19         ux, uy = points[u]
20         for v in range(n):
21             if not in_mst[v]:
22                 vx, vy = points[v]
23                 heapq.heappush(heap, (abs(ux - vx) + abs(uy - vy), v))
24     return total

```

**** Error-prone spots:****

- Add a point's cost only when it is first popped and marked in-tree; summing on push double-counts.
- Skip stale heap entries (if in_mst[u]: continue) — each point is pushed once per prior addition, so duplicates are normal.
- Seed exactly one point with cost 0; seeding several or with nonzero cost corrupts the total.
- Stop after n points are in the tree; a spanning tree has exactly $n-1$ edges, so the loop must

terminate cleanly.

- › Manhattan distance uses absolute values on both coordinates; Euclidean or signed differences give wrong costs.

Follow-up. Kruskal's algorithm is the alternative: generate all n^2 candidate edges, sort by cost, and union endpoints with DSU, skipping edges that would form a cycle — $O(n^2 \log n)$ dominated by the sort, and naturally cheaper to reason about for sparse graphs. For very dense graphs the eager $O(n^2)$ array-based Prim's (no heap) is asymptotically best.

5.10 Critical Connections in a Network / Bridges (LC 1192)

Problem. A network of n servers (0-indexed) connected by undirected connections. A *critical connection* (bridge) is an edge whose removal disconnects some servers from others. Return all critical connections in any order.

Algorithm — Tarjan's bridge-finding via DFS discovery/low times · Time $O(V + E)$, Space $O(V + E)$. A single DFS computes, for each node, its discovery time $disc[u]$ and its $low[u]$ = the smallest discovery time reachable from u 's subtree via tree edges plus at most one back edge. An edge (u, v) (v a DFS child) is a bridge exactly when $low[v] > disc[u]$: the subtree rooted at v has no back edge climbing to u or above, so that edge is the only link. This is the standard linear bridge algorithm.

Idea. Run an iterative DFS assigning increasing discovery times. $low[u]$ starts as $disc[u]$ and is lowered by (a) $disc$ of any back-edge target and (b) low of each tree child after recursion returns. After finishing a child v , if $low[v] > disc[u]$ the tree edge (u, v) is a bridge. Crucially, skip only the *single* edge back to the parent — but skip it by edge identity, not by vertex, to handle parallel edges correctly.

```

1 def criticalConnections(n: int, connections: list[list[int]]) → list[list[int]]:
2     graph = [[] for _ in range(n)]
3     for u, v in connections:
4         graph[u].append(v)
5         graph[v].append(u)
6
7     disc = [-1] * n          # discovery time, -1 = unvisited
8     low = [0] * n
9     bridges = []
10    timer = 0
11
12    # iterative DFS; child_idx tracks next neighbor to explore per frame
13    for start in range(n):
14        if disc[start] != -1:
15            continue
16        stack = [(start, -1, 0)] # (node, parent, child_index)
17        while stack:
18            u, parent, ci = stack[-1]
19            if ci == 0:
20                disc[u] = low[u] = timer
21                timer += 1
22            if ci < len(graph[u]):
23                stack[-1] = (u, parent, ci + 1) # advance pointer
24                v = graph[u][ci]
25                if v == parent:
26                    continue # skip the edge back to parent
27                if disc[v] == -1:
28                    stack.append((v, u, 0)) # recurse into unvisited child

```

```

29         else:
30             low[u] = min(low[u], disc[v]) # back edge
31     else:
32         stack.pop() # done with u
33         if parent != -1:
34             low[parent] = min(low[parent], low[u])
35             if low[u] > disc[parent]: # bridge condition
36                 bridges.append([parent, u])
37     return bridges

```

**** Error-prone spots:****

- The bridge test is strict $\text{low}[v] > \text{disc}[u]$ (equality means a back edge reaches u , so *not* a bridge); using $\geq$ over-reports.
- For back edges update $\text{low}[u] = \min(\text{low}[u], \text{disc}[v])$ using $\text{disc}[v]$, *not* $\text{low}[v]$ — using $\text{low}[v]$ is a classic bug that can miss bridges.
- Propagate $\text{low}[u]$ up to the parent *after* finishing u ($\text{low}[\text{parent}] = \min(\text{low}[\text{parent}], \text{low}[u])$); forgetting this breaks the recurrence.
- Skip the parent edge, but if parallel edges (two edges between the same pair) are allowed, skip by edge index, not by parent vertex, else you wrongly treat a real second edge as the parent link.
- Initialize disc to -1 (unvisited sentinel) and loop over all start nodes for graphs that may be disconnected.
- Bridges have no notion of direction here; $[\text{parent}, u]$ and $[u, \text{parent}]$ are the same edge — return either.

Follow-up. The sibling problem is *articulation points* (cut vertices): node u is a cut vertex if it is the DFS root with ≥ 2 children, or a non-root with some child v where $\text{low}[v] \geq \text{disc}[u]$ (note the $\geq$ here, unlike bridges). Tarjan's strongly-connected-components algorithm reuses the same disc/low machinery on directed graphs.

6 Dynamic Programming

6.1 Climbing Stairs (LC 70)

Problem. You climb a staircase of n steps. Each move you ascend either 1 or 2 steps. Count the number of distinct ways to reach the top.

Optimal — DP · Time $O(n)$, Space $O(1)$. A naive recursion is $O(2^n)$; the 1D table is $O(n)$ space, but only the last two values matter, so two scalars suffice.

State & recurrence. Let $\text{dp}[i]$ = number of distinct ways to reach step i . Transition: $\text{dp}[i] = \text{dp}[i-1] + \text{dp}[i-2]$ (last move was a 1-step from $i-1$ or a 2-step from $i-2$). Base cases: $\text{dp}[0] = 1$ (one empty way to stand at the ground), $\text{dp}[1] = 1$. Answer: $\text{dp}[n]$.

Idea. Every path to step i ends with exactly one final hop of size 1 or 2; the two end-states partition all paths, so their counts add. This is the Fibonacci recurrence shifted by one.

```

1 def climb_stairs(n: int) → int:
2     if n ≤ 1:
3         return 1
4     prev2, prev1 = 1, 1          # dp[0], dp[1]
5     for _ in range(2, n + 1):
6         prev2, prev1 = prev1, prev1 + prev2
7     return prev1

```

**** Error-prone spots:****

- › $dp[0] = 1$, not 0 — there is exactly one way to be at the start (do nothing).
- › Handle $n = 0$ and $n = 1$ before the loop or the loop body never runs and you return the wrong scalar.
- › The two-variable swap must be simultaneous ($prev2, prev1 = prev1, prev1 + prev2$); doing it in two statements clobbers $prev1$ first.
- › Loop range is $range(2, n + 1)$ — inclusive of n ; $range(2, n)$ is an off-by-one undercount.
- › This is Fibonacci, so it overflows fixed-width ints in other languages; Python's bigints are fine.

6.2 Coin Change (LC 322)

Problem. Given coin denominations `coins` and a target amount, return the fewest number of coins that sum to `amount`, or -1 if it is impossible. You have an unlimited supply of each coin.

Optimal — DP · Time $O(\text{amount} * \text{len}(\text{coins}))$, Space $O(\text{amount})$. This is unbounded-knapsack-shaped; a 1D array indexed by sub-amount is optimal.

State & recurrence. Let $dp[a]$ = minimum coins to make amount a . Transition: $dp[a] = \min(dp[a - c] + 1)$ over all coins $c \leq a$. Base case: $dp[0] = 0$; initialize all other cells to INF (use $\text{amount} + 1$ as a safe sentinel since no answer can exceed `amount`). Answer: $dp[\text{amount}]$ if it is $< INF$, else -1.

Idea. An optimal solution for a uses some last coin c ; remove it and the remainder is an optimal solution for $a - c$. Trying every possible last coin and taking the minimum guarantees optimality (optimal substructure).

```

1 def coin_change(coins: list[int], amount: int) → int:
2     INF = amount + 1
3     dp = [0] + [INF] * amount          # dp[0] = 0
4     for a in range(1, amount + 1):
5         for c in coins:
6             if c ≤ a and dp[a - c] + 1 < dp[a]:
7                 dp[a] = dp[a - c] + 1
8     return dp[amount] if dp[amount] ≠ INF else -1

```

**** Error-prone spots:****

- › This is a *min* problem, so initialize unreachable cells to a large sentinel, not 0; $\text{amount} + 1$ strictly exceeds any real answer and never overflows.
- › Forgetting the final "if unreachable return -1" check returns the sentinel.
- › Because coins are unbounded, the inner loop iterates amounts *forward* (each coin may be reused); reversing would convert it to a 0/1 (one-coin-each) problem.

- › $dp[0] = 0$, not INF — zero coins make amount 0.
- › Guard $c \leq a$ before indexing $dp[a - c]$, otherwise you index negative or wrap-around.
- › Greedy (always take the largest coin) is wrong for arbitrary denominations (e.g. coins $[1, 3, 4]$, amount 6: greedy gives $4+1+1=3$, DP gives $3+3=2$).

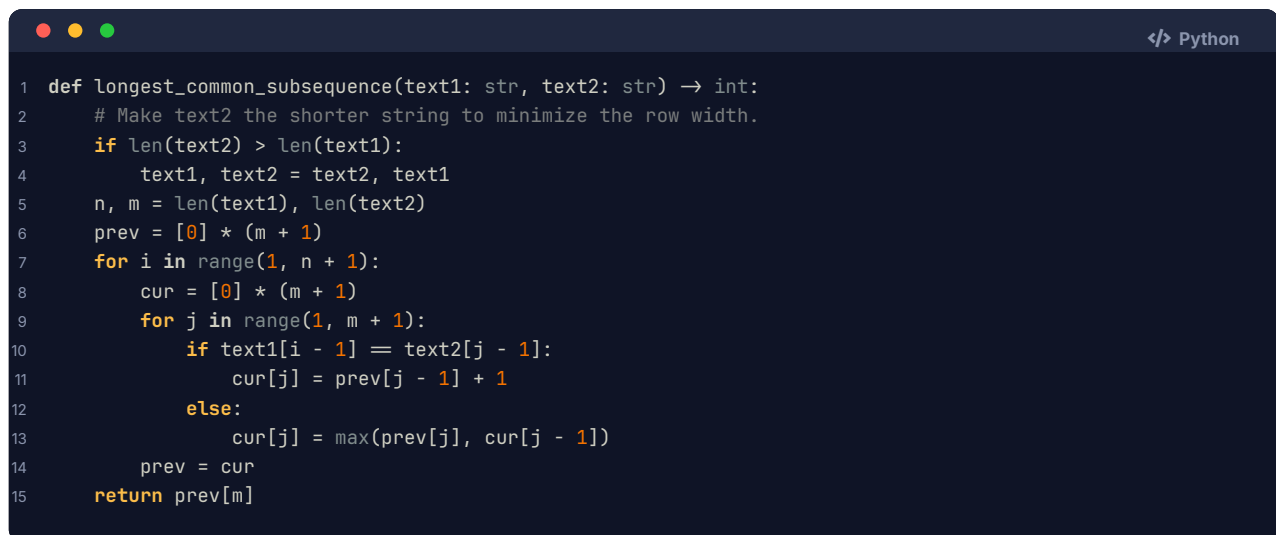
6.3 Longest Common Subsequence (LC 1143)

Problem. Given strings `text1` and `text2`, return the length of their longest common subsequence (characters in the same relative order, not necessarily contiguous).

Optimal — DP · Time $O(n*m)$, Space $O(\min(n,m))$. Full table is $O(n*m)$ space; since each row depends only on the previous row, two rolling rows reduce it to $O(\min(n,m))$.

State & recurrence. Let $dp[i][j]$ = LCS length of prefixes `text1[:i]` and `text2[:j]`. Transition: if `text1[i-1] == text2[j-1]` then $dp[i][j] = dp[i-1][j-1] + 1$; else $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$. Base cases: $dp[0][*] = dp[*][0] = 0$ (empty prefix). Answer: $dp[n][m]$.

Idea. Compare the last characters of the two prefixes. If they match, they can both end the LCS, adding 1 to the LCS of the shorter prefixes. If not, the LCS must drop at least one of the two last characters, so take the better of the two reduced subproblems.



```

1 def longest_common_subsequence(text1: str, text2: str) -> int:
2     # Make text2 the shorter string to minimize the row width.
3     if len(text2) > len(text1):
4         text1, text2 = text2, text1
5     n, m = len(text1), len(text2)
6     prev = [0] * (m + 1)
7     for i in range(1, n + 1):
8         cur = [0] * (m + 1)
9         for j in range(1, m + 1):
10             if text1[i - 1] == text2[j - 1]:
11                 cur[j] = prev[j - 1] + 1
12             else:
13                 cur[j] = max(prev[j], cur[j - 1])
14         prev = cur
15     return prev[m]

```

**** Error-prone spots:****

- › The DP indices are 1-based over a $(n+1) \times (m+1)$ table, but string indices are 0-based: compare `text1[i-1]` with `text2[j-1]`.
- › This computes *subsequence*, not *substring*; the `max` branch (rather than reset to 0) is what allows gaps.
- › In the rolling version, `prev[j-1]` is the diagonal ($dp[i-1][j-1]$) and `cur[j-1]` is the left ($dp[i][j-1]$); mixing them up corrupts the recurrence.
- › Row 0 and column 0 must be all zeros — these encode the empty-prefix base case.
- › To also recover the actual subsequence you need the full $O(n*m)$ table to backtrack; the rolling array only yields the length.

6.4 0/1 Knapsack

Problem. Given n items with weights $w[i]$ and values $v[i]$, and a knapsack capacity W , choose a subset (each item at most once) maximizing total value subject to total weight $\leq W$.

Optimal — DP · Time $O(n*W)$, Space $O(W)$. 2D table is $O(n*W)$ space; a single rolling 1D array

of length $W+1$ suffices if the capacity loop runs in reverse.

State & recurrence. Let $dp[i][c]$ = max value using the first i items with capacity c . Transition: $dp[i][c] = \max(dp[i-1][c], dp[i-1][c - w[i]] + v[i])$ when $w[i] \leq c$, else $dp[i][c] = dp[i-1][c]$. Base cases: $dp[0][c] = 0$. Answer: $dp[n][W]$.

Idea. For each item decide include-or-exclude. Excluding leaves $dp[i-1][c]$; including consumes $w[i]$ capacity and adds $v[i]$ on top of the best solution for the *previous* items at the reduced capacity. The reference $dp[i-1][\dots]$ is what forbids reusing the item.

```

1 def knapsack_01(weights: list[int], values: list[int], W: int) → int:
2     dp = [0] * (W + 1)
3     for i in range(len(weights)):
4         w, v = weights[i], values[i]
5         for c in range(W, w - 1, -1):      # REVERSE: capacity high → low
6             dp[c] = max(dp[c], dp[c - w] + v)
7     return dp[W]

```

**** Error-prone spots:****

- **Iterate capacity in REVERSE** (`range(W, w-1, -1)`). The 1D array overwrites $dp[c]$ in place; reading $dp[c - w]$ must still hold the *previous item's* value, i.e. the state before this item was considered. Reverse order guarantees $dp[c - w]$ (a smaller index) has not yet been updated this round, so each item is used at most once.
- A forward loop here would let $dp[c - w]$ already include the current item, silently turning this into *unbounded* knapsack.
- Loop lower bound is $w - 1$ so the smallest c processed is w ; for $c < w$ the item cannot fit and $dp[c]$ stays.
- This is a *maximization*; initialize to 0 (empty knapsack value), not $-\infty$, unless you must exactly fill the bag.
- If you need to enforce exact capacity filling, initialize $dp[1..W] = -\infty$ and only $dp[0] = 0$.

6.5 Unbounded Knapsack

Problem. Same as 0/1 knapsack, but each item may be chosen an unlimited number of times. Maximize value with total weight $\leq W$.

Optimal — DP · Time $O(n \cdot W)$, Space $O(W)$. One rolling 1D array; the only change from 0/1 is the capacity loop direction.

State & recurrence. Let $dp[c]$ = max value achievable with capacity c using any items, any counts. Transition: $dp[c] = \max(dp[c], dp[c - w[i]] + v[i])$ for each item with $w[i] \leq c$. Base case: $dp[0] = 0$. Answer: $dp[W]$.

Idea. Because an item can repeat, when we include item i at capacity c we are allowed to reference a state at $c - w[i]$ that *already contains item i* . Iterating capacity forward makes exactly that "already updated" value visible, so repeats are naturally permitted.

```

1 def knapsack_unbounded(weights: list[int], values: list[int], W: int) → int:
2     dp = [0] * (W + 1)
3     for c in range(1, W + 1):
4         for i in range(len(weights)):

```

```

5         w, v = weights[i], values[i]
6         if w ≤ c:
7             dp[c] = max(dp[c], dp[c - w] + v)
8     return dp[W]

```

**** Error-prone spots:****

- ▶ **Iterate capacity FORWARD** (`range(1, W+1)`). Forward order means `dp[c - w]` may already reflect using item `i`, which is exactly the reuse we want — the mirror image of the 0/1 reverse loop.
- ▶ The loop nesting can be either order (capacity-outer/item-inner or item-outer/capacity-inner) for the *max-value* variant; both are correct. But for *counting combinations* the order matters (item-outer avoids double counting permutations), so keep the distinction in mind.
- ▶ Still guard `w ≤ c` before indexing `dp[c - w]`.
- ▶ Confusing this with 0/1 is the single most common knapsack bug: the recurrence text is identical; only the iteration direction differs.

6.6 Longest Increasing Subsequence (LC 300)

Problem. Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

Optimal — DP · Time $O(n \log n)$, Space $O(n)$. A clean $O(n^2)$ DP exists; patience sorting with binary search gives $O(n \log n)$.

State & recurrence. $O(n^2)$ DP: let `dp[i]` = length of the longest increasing subsequence *ending exactly at index i*. Transition: `dp[i] = 1 + max(dp[j] for j < i if nums[j] < nums[i])`, or 1 if no such `j`. Base case: every `dp[i]` starts at 1 (the element alone). Answer: `max(dp)`. $O(n \log n)$: maintain `tails`, where `tails[k]` = smallest possible tail value of an increasing subsequence of length `k+1`. For each `x`, binary-search the first `tails[k] ≥ x` and overwrite it (or append if `x` exceeds all tails). Answer: `len(tails)`.

Idea. Any LIS ending at `i` extends some shorter LIS ending at an earlier, smaller element. The `tails` array works because keeping the *smallest* tail for each length is greedily optimal: a smaller tail can never block a future extension that a larger tail would allow. `tails` is always sorted, enabling binary search. (`tails` is not itself a valid subsequence, only its length is meaningful.)

```

1 from bisect import bisect_left
2
3 def length_of_lis(nums: list[int]) → int:
4     tails = []
5     for x in nums:
6         i = bisect_left(tails, x)    # first tail ≥ x (strict increase)
7         if i == len(tails):
8             tails.append(x)          # x extends the longest run
9         else:
10            tails[i] = x              # tighten the tail of length i+1
11    return len(tails)
12
13 def length_of_lis_n2(nums: list[int]) → int:
14     if not nums:
15         return 0
16     dp = [1] * len(nums)

```

```

17     for i in range(len(nums)):
18         for j in range(i):
19             if nums[j] < nums[i]:
20                 dp[i] = max(dp[i], dp[j] + 1)
21     return max(dp)

```

**** Error-prone spots:****

- *Strict* increase uses `bisect_left` (replaces the first element $\geq x$). For *non-decreasing* (allow equal) use `bisect_right` instead — this swap is a classic trap.
- The tails array's contents are not a real LIS; only `len(tails)` is the answer. Do not try to read the subsequence off it directly.
- In the $O(n^2)$ version every `dp[i]` must start at 1 (the single element), not 0.
- The answer is `max(dp)`, not `dp[-1]` — the LIS need not end at the last element.
- Empty input: return 0; guard before `max(dp)`.
- To reconstruct the actual subsequence in the $O(n \log n)$ version, store predecessor indices and the position each `x` was placed.

Follow-up. Number of LIS (LC 673) needs a parallel `count[i]` array; Russian-doll envelopes (LC 354) reduces to LIS after sorting one dimension ascending and the other descending to forbid ties.

6.7 Edit Distance (LC 72)

Problem. Given two strings `word1` and `word2`, return the minimum number of single-character operations (insert, delete, replace) to transform `word1` into `word2`.

Optimal — DP · Time $O(n \cdot m)$, Space $O(\min(n, m))$. Full table $O(n \cdot m)$; rolling two rows gives $O(\min(n, m))$.

State & recurrence. Let `dp[i][j]` = edit distance between `word1[:i]` and `word2[:j]`. Transition: if `word1[i-1] == word2[j-1]` then `dp[i][j] = dp[i-1][j-1]` (free match); else `dp[i][j] = 1 + min(dp[i-1][j-1] (replace), dp[i-1][j] (delete), dp[i][j-1] (insert))`. Base cases: `dp[0][j] = j` (insert `j` chars), `dp[i][0] = i` (delete `i` chars). Answer: `dp[n][m]`.

Idea. Align the last characters. If they match, no cost and recurse on both shortened prefixes. Otherwise one of three edits resolves the mismatch: replace (advance both), delete from `word1` (advance `i`), or insert into `word1` (advance `j`); take the cheapest.

```

1 def min_distance(word1: str, word2: str) → int:
2     n, m = len(word1), len(word2)
3     prev = list(range(m + 1))      # dp[0][j] = j
4     for i in range(1, n + 1):
5         cur = [i] + [0] * m        # cur[0] = dp[i][0] = i
6         for j in range(1, m + 1):
7             if word1[i - 1] == word2[j - 1]:
8                 cur[j] = prev[j - 1]
9             else:
10                cur[j] = 1 + min(prev[j - 1], prev[j], cur[j - 1])
11        prev = cur
12    return prev[m]

```

**** Error-prone spots:****

- › The two base rows/columns are NOT zero: $dp[0][j] = j$ and $dp[i][0] = i$. Forgetting these (leaving zeros) silently lets transformations be free.
- › In the rolling version, $cur[0]$ must be reset to i each iteration, not 0.
- › Replace, delete, insert map to diagonal, up, left respectively: $prev[j-1]$, $prev[j]$, $cur[j-1]$. Mislabeling them is easy and gives a wrong-but-plausible answer.
- › On a character *match* the cost adds 0 and you take only $prev[j-1]$ — do not also $\min$ with the other neighbors (that can incorrectly lower the value).
- › String indices are $i-1$, $j-1$ against 1-based DP indices.

6.8 Regular Expression Matching (LC 10)

Problem. Implement regex matching for `'.'` (matches any single character) and `'*'` (matches zero or more of the *preceding* element). The match must cover the *entire* input string s against pattern p .

Optimal — DP · Time $O(nm)$, Space $O(nm)$. $n = \text{len}(s)$, $m = \text{len}(p)$; can be rolled to $O(m)$ but the 2D table is clearest and the standard interview answer.

State & recurrence. Let $dp[i][j]$ = does $s[:i]$ match $p[:j]$. Transitions:

- › If $p[j-1] = '*'$: $dp[i][j] = dp[i][j-2]$ (use the $x*$ as zero occurrences) OR, if $p[j-2]$ matches $s[i-1]$ (i.e. $p[j-2] = s[i-1]$ or $p[j-2] = '.'$), also $dp[i-1][j]$ (consume one more s char, keep $x*$).
- › Else (ordinary char or `.`): $dp[i][j] = dp[i-1][j-1]$ AND $p[j-1]$ matches $s[i-1]$.

Base cases: $dp[0][0] = \text{True}$; $dp[0][j] = dp[0][j-2]$ when $p[j-1] = '*'$ (patterns like $a*b*c*$ match the empty string). Answer: $dp[n][m]$.

Idea. A `*` together with its preceding token is a unit meaning "zero or more". "Zero" skips two pattern chars ($dp[i][j-2]$); "one or more" consumes a matching s char while leaving the $x*$ in play ($dp[i-1][j]$). All other positions match exactly one character.

```

1 def is_match(s: str, p: str) → bool:
2     n, m = len(s), len(p)
3     dp = [[False] * (m + 1) for _ in range(n + 1)]
4     dp[0][0] = True
5     # Empty string vs patterns that can vanish: a*, a*b*, ...
6     for j in range(1, m + 1):
7         if p[j - 1] == '*':
8             dp[0][j] = dp[0][j - 2]
9         for i in range(1, n + 1):
10            for j in range(1, m + 1):
11                if p[j - 1] == '*':
12                    # zero occurrences of the char before '*'
13                    dp[i][j] = dp[i][j - 2]
14                    # one-or-more, if preceding pattern char matches s[i-1]
15                    if p[j - 2] == s[i - 1] or p[j - 2] == '.':
16                        dp[i][j] = dp[i][j] or dp[i - 1][j]
17                else:
18                    if p[j - 1] == s[i - 1] or p[j - 1] == '.':
19                        dp[i][j] = dp[i - 1][j - 1]
20    return dp[n][m]

```

**** Error-prone spots:****

- › The `*` always refers to $p[j-2]$ (the char *before* it); "zero occurrences" jumps to $dp[i][j-2]$, not

`dp[i][j-1]`.

- › Don't forget the empty-string row: `dp[0][j] = dp[0][j-2]` for `*`, which lets `a*` etc. match `""`.
- › The "one or more" branch (`dp[i-1][j]`) is only valid when `p[j-2]` matches `s[i-1]`; guarding this is essential.
- › `.` matches any single char but NOT the empty string; `.*` together matches anything including empty.
- › A leading `*` is invalid input by problem constraints, so `p[0]` is never `'*'`; still, never index `p[j-2]` when `j < 2`.
- › The match must be *full*: the answer is `dp[n][m]`, requiring both string and pattern fully consumed.

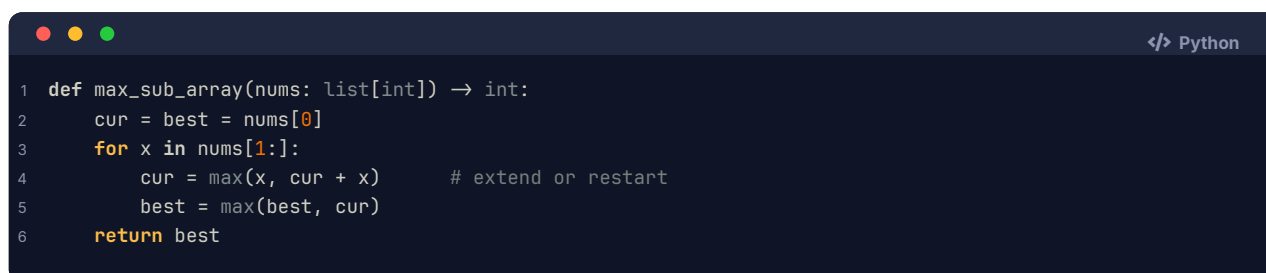
6.9 Maximum Subarray --- Kadane's (LC 53)

Problem. Given an integer array `nums`, find the contiguous subarray (at least one element) with the largest sum and return that sum.

Optimal — DP · Time $O(n)$, Space $O(1)$. Kadane's algorithm is a 1D DP collapsed to two scalars.

State & recurrence. Let `cur[i]` = maximum sum of a subarray *ending exactly at index i*. Transition: `cur[i] = max(nums[i], cur[i-1] + nums[i])` — either start fresh at `i`, or extend the best subarray ending at `i-1`. Base case: `cur[0] = nums[0]`. Answer: `max(cur[i])` over all `i` (track a running global best).

Idea. A best subarray ending at `i` either is just `nums[i]` or appends `nums[i]` to the best subarray ending at `i-1`. If that previous best was negative, dropping it (starting fresh) is strictly better. The overall answer is the max across all ending positions.



```

1 def max_sub_array(nums: list[int]) -> int:
2     cur = best = nums[0]
3     for x in nums[1:]:
4         cur = max(x, cur + x)      # extend or restart
5         best = max(best, cur)
6     return best

```

**** Error-prone spots:****

- › Initialize both `cur` and `best` to `nums[0]`, NOT 0 — an all-negative array (e.g. `[-3, -1, -2]`) must return `-1`, and a 0 seed would wrongly return 0.
- › `best` must be updated every step, including when `cur` decreases; it is not simply the final `cur`.
- › The subarray must be non-empty; do not allow an "empty subarray of sum 0" unless the problem explicitly permits it.
- › The `max(x, cur + x)` order encodes "restart vs extend"; reversing operands or using `cur` instead of `x` for restart breaks the all-negative case.

Follow-up. To recover the indices, track the start when you restart (`cur = x`) and the end when you update `best`.

6.10 Maximum Product Subarray (LC 152)

Problem. Given an integer array `nums`, find the contiguous subarray with the largest *product* and return that product.

Optimal — DP · Time $O(n)$, Space $O(1)$. Track both the max and min product ending at each

index.

State & recurrence. Let $mx[i]$ = max product of a subarray ending at i , and $mn[i]$ = min product ending at i . Transitions: $mx[i] = \max(nums[i], mx[i-1]*nums[i], mn[i-1]*nums[i])$ and $mn[i] = \min(nums[i], mx[i-1]*nums[i], mn[i-1]*nums[i])$. Base: $mx[0] = mn[0] = nums[0]$. Answer: $\max(mx[i])$ over all i .

Idea. A negative number flips sign: multiplying it by the most *negative* prefix product yields the largest positive product. So we must carry the running minimum alongside the maximum, since either can become the new maximum after a sign flip. Zeros reset both to the current element.

```

1 def max_product(nums: List[int]) -> int:
2     cur_max = cur_min = best = nums[0]
3     for x in nums[1:]:
4         cand = (x, cur_max * x, cur_min * x)
5         cur_max = max(cand)
6         cur_min = min(cand)      # use old cur_max/cur_min from same step
7         best = max(best, cur_max)
8     return best

```

**** Error-prone spots:****

- You MUST compute the candidate products from the *previous* cur_max/cur_min before reassigning either. Updating cur_max first, then using it for cur_min , is a classic bug — compute both from a shared $cand$ tuple (or stash the old cur_max in a temp).
- Include x itself as a candidate (the “restart” option), exactly as in Kadane's.
- Zeros are handled automatically: cur_max and cur_min both collapse toward x (zero), correctly cutting the subarray.
- Seed all three trackers with $nums[0]$, not 1 or 0; a single-element negative array must return that negative value.
- Unlike sum, you cannot drop the min: the largest product can come from two negatives.

6.11 House Robber (LC 198)

Problem. Houses in a row hold $nums[i]$ money each. You cannot rob two *adjacent* houses (alarms link neighbors). Return the maximum total you can rob.

Optimal — DP · Time $O(n)$, Space $O(1)$. 1D DP collapsed to two scalars.

State & recurrence. Let $dp[i]$ = max money robbing among houses $0..i$. Transition: $dp[i] = \max(dp[i-1]$ (skip house i), $dp[i-2] + nums[i]$ (rob house i)). Base cases: $dp[0] = nums[0]$, $dp[1] = \max(nums[0], nums[1])$. Answer: $dp[n-1]$.

Idea. At house i you either skip it (keep $dp[i-1]$) or rob it, which forbids $i-1$ and adds $nums[i]$ to the best of $0..i-2$. The two options are mutually exclusive and exhaustive, so take the max.

```

1 def rob(nums: List[int]) -> int:
2     prev2, prev1 = 0, 0      # dp[i-2], dp[i-1]
3     for x in nums:
4         prev2, prev1 = prev1, max(prev1, prev2 + x)
5     return prev1

```

**** Error-prone spots:****

- › Seeding both scalars to 0 cleanly handles $n = 0$ and $n = 1$ without special cases (the recurrence absorbs the base cases).
- › The simultaneous tuple update is required: compute $\max(\text{prev1}, \text{prev2} + x)$ using the *old* prev1 and prev2 .
- › "Rob house i " adds $\text{dp}[i-2]$, not $\text{dp}[i-1]$ — using $i-1$ would allow robbing adjacent houses.
- › Don't confuse the "max so far" interpretation; $\text{dp}[i]$ already encodes the optimal choice for the prefix, monotonically non-decreasing.

6.12 House Robber II (LC 213, circular)

Problem. Same as House Robber, but the houses are arranged in a *circle*: the first and last houses are adjacent. Return the maximum robbery.

Optimal — DP · Time $O(n)$, Space $O(1)$. Run the linear House Robber twice on two disjoint windows and take the max.

State & recurrence. Because house 0 and house $n-1$ are now adjacent, they cannot both be robbed. Split into two non-circular subproblems: (a) houses 0 .. $n-2$ (allow first, exclude last) and (b) houses 1 .. $n-1$ (exclude first, allow last). Each is solved by the linear recurrence $\text{dp}[i] = \max(\text{dp}[i-1], \text{dp}[i-2] + \text{nums}[i])$. Answer: $\max(\text{rob_linear}(\text{nums}[:-1]), \text{rob_linear}(\text{nums}[1:]))$. Special-case $n = 1$: answer is $\text{nums}[0]$.

Idea. The only new constraint couples the two endpoints. Forbidding at least one of them removes the wrap-around edge, reducing the circle to a line. Whichever of "drop the last" or "drop the first" yields more is optimal, and every valid solution falls into at least one window.



```

1 def rob_circular(nums: list[int]) → int:
2     if len(nums) == 1:
3         return nums[0]
4
5     def rob_linear(houses) → int:
6         prev2, prev1 = 0, 0
7         for x in houses:
8             prev2, prev1 = prev1, max(prev1, prev2 + x)
9         return prev1
10
11    return max(rob_linear(nums[:-1]), rob_linear(nums[1:]))

```

**** Error-prone spots:****

- › The $n = 1$ case must be special-cased: $\text{nums}[:-1]$ and $\text{nums}[1:]$ are both empty, which would return 0 instead of $\text{nums}[0]$.
- › The two windows are $\text{nums}[0:n-1]$ and $\text{nums}[1:n]$; using overlapping or wrong slices breaks the adjacency guarantee.
- › It is *not* enough to just forbid robbing both endpoints inside one pass — the cleanest correct reduction is two independent linear runs.
- › For $n = 2$, both slices are single elements and $\max$ correctly returns the larger house.

6.13 Unique Paths (LC 62)

Problem. A robot sits at the top-left of an $m \times n$ grid and may move only right or down. Count the distinct paths to the bottom-right cell.

Optimal – DP · Time $O(m \cdot n)$, Space $O(n)$. Full grid $O(m \cdot n)$; rolling a single row gives $O(n)$. (Closed form: $C(m+n-2, m-1)$.)

State & recurrence. Let $dp[i][j]$ = number of paths to cell (i, j) . Transition: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$ (arrive from above or from the left). Base cases: $dp[0][j] = 1$ for all j and $dp[i][0] = 1$ for all i (only one straight-line path along an edge). Answer: $dp[m-1][n-1]$.

Idea. Every path into (i, j) makes its last move either downward from $(i-1, j)$ or rightward from $(i, j-1)$; these two sets are disjoint and exhaustive, so path counts add.

```

1 def unique_paths(m: int, n: int) → int:
2     row = [1] * n          # top row: all 1s
3     for _ in range(1, m):
4         for j in range(1, n):
5             row[j] += row[j - 1] # row[j]=above (old) + row[j-1]=left (new)
6     return row[n - 1]

```

**** Error-prone spots:****

- › Initialize the entire first row (and conceptually first column) to 1 — there is exactly one edge-hugging path.
- › In the rolling-row version, `row[j] += row[j-1]` works because at that moment `row[j]` still holds the value from the row above (not yet updated) and `row[j-1]` already holds this row's left neighbor.
- › Inner loop starts at $j = 1$; `row[0]` stays 1 throughout (the left column).
- › Answer index is $(m-1, n-1)$, zero-based; mixing up m (rows) and n (cols) is a frequent slip.

6.14 Minimum Path Sum (LC 64)

Problem. Given an $m \times n$ grid of non-negative numbers, find a path from top-left to bottom-right that minimizes the sum of values along the path, moving only right or down. Return that minimum sum.

Optimal – DP · Time $O(m \cdot n)$, Space $O(n)$. Rolling a single row gives $O(n)$ space; can also be done in place over the grid.

State & recurrence. Let $dp[i][j]$ = minimum path-sum to reach (i, j) . Transition: $dp[i][j] = \text{grid}[i][j] + \min(dp[i-1][j], dp[i][j-1])$. Base cases: $dp[0][0] = \text{grid}[0][0]$; first row $dp[0][j] = dp[0][j-1] + \text{grid}[0][j]$; first column $dp[i][0] = dp[i-1][0] + \text{grid}[i][0]$ (only one way to reach an edge cell). Answer: $dp[m-1][n-1]$.

Idea. The cheapest way to stand on (i, j) is its own cost plus the cheaper of the two cells that can precede it. Optimal substructure holds because all weights are non-negative and moves are monotone (right/down only).

```

1 def min_path_sum(grid: list[list[int]]) → int:
2     m, n = len(grid), len(grid[0])
3     row = [0] * n

```

```

4     row[0] = grid[0][0]
5     for j in range(1, n):           # first row prefix sums
6         row[j] = row[j - 1] + grid[0][j]
7     for i in range(1, m):
8         row[0] += grid[i][0]         # first column
9         for j in range(1, n):
10            row[j] = grid[i][j] + min(row[j], row[j - 1])
11            # row[j] (old) = cell above; row[j-1] (new) = cell to the left
12     return row[n - 1]

```

**** Error-prone spots:****

- › The first row and first column have NO min — there is only one predecessor; they are pure prefix sums. Applying min against an uninitialized 0 there understates the cost.
- › In the rolling version, `row[0] += grid[i][0]` must run *before* the inner loop each row, updating the left edge.
- › Inside the loop, `row[j]` (pre-update) is the cell above and `row[j-1]` (post-update) is the cell to the left.
- › Always add `grid[i][j]` — every cell's own value is included, including the start and end cells.

6.15 Burst Balloons (LC 312)

Problem. You have n balloons with values `nums[i]`. Bursting balloon i earns `nums[left] * nums[i] * nums[right]`, where `left/right` are the *currently adjacent* balloons (out-of-range neighbors count as value 1). After bursting, neighbors become adjacent. Maximize total coins over all burst orders.

Optimal — Interval DP · Time $O(n^3)$, Space $O(n^2)$. Pad the array with sentinel 1s on both ends.

State & recurrence. Pad to `a = [1] + nums + [1]`. Let `dp[i][j]` = max coins from bursting all balloons *strictly between* indices i and j (exclusive) in `a`. Transition: pick the balloon k ($i < k < j$) to burst **last** in that open interval: `dp[i][j] = max over k of ( dp[i][k] + a[i]*a[k]*a[j] + dp[k][j] )`. Base case: `dp[i][j] = 0` when $j - i < 2$ (no interior balloon). Answer: `dp[0][len(a)-1]`.

Idea. The trick is to fix the *last* balloon to burst in an interval. When k bursts last, its neighbors are exactly the interval's untouched boundaries `a[i]` and `a[j]` (everything else inside is already gone), so its gain is the clean product `a[i]*a[k]*a[j]`, and the two sides `dp[i][k]` and `dp[k][j]` are independent subproblems. Choosing "first to burst" instead would couple the subproblems and fail.

```

1 def max_coins(nums: list[int]) -> int:
2     a = [1] + nums + [1]
3     n = len(a)
4     dp = [[0] * n for _ in range(n)]
5     # length = gap between boundaries i and j (must be ≥ 2 to contain a balloon)
6     for length in range(2, n):
7         for i in range(0, n - length):
8             j = i + length
9             best = 0
10            for k in range(i + 1, j):
11                coins = dp[i][k] + a[i] * a[k] * a[j] + dp[k][j]

```

```

12         if coins > best:
13             best = coins
14         dp[i][j] = best
15     return dp[0][n - 1]

```

**** Error-prone spots:****

- › Think **last to burst**, not first — this is what makes the boundary multipliers $a[i]$ and $a[j]$ fixed and the two halves independent.
- › The padding sentinels (value 1) model the "out of bounds = multiply by 1" rule; the answer interval is the whole padded array $dp[0][n-1]$.
- › $dp[i][j]$ is *open* (balloons strictly between i and j); the boundaries i and j themselves are NOT burst within this subproblem.
- › Iterate by increasing interval *length* so that $dp[i][k]$ and $dp[k][j]$ (shorter intervals) are already computed.
- › The product uses $a[i]*a[k]*a[j]$ — the interval *endpoints*, not $a[k-1]/a[k+1]$, because everything strictly inside is gone by the time k bursts.
- › k ranges over $i+1 \dots j-1$; an interval with $j - i < 2$ has no interior and contributes 0.

6.16 Word Break (LC 139)

Problem. Given a string s and a dictionary `wordDict`, determine whether s can be segmented into a space-separated sequence of one or more dictionary words (words may be reused).

Optimal — DP · Time $O(n^2 * L)$ worst (or $O(n^2)$ with hashing on slice), Space $O(n)$. Store the dictionary in a set for $O(1)$ membership.

State & recurrence. Let $dp[i]$ = can $s[:i]$ be fully segmented. Transition: $dp[i] = \text{True}$ if there exists a split point $j < i$ with $dp[j] = \text{True}$ and $s[j:i]$ in the word set. Base case: $dp[0] = \text{True}$ (the empty prefix is trivially segmentable). Answer: $dp[n]$.

Idea. A prefix of length i is breakable iff some shorter breakable prefix of length j is followed by a dictionary word $s[j:i]$. We scan all earlier split points; one valid (prefix, word) pair suffices.

```

1 def word_break(s: str, word_dict: list[str]) → bool:
2     words = set(word_dict)
3     n = len(s)
4     max_len = max((len(w) for w in words), default=0)
5     dp = [False] * (n + 1)
6     dp[0] = True
7     for i in range(1, n + 1):
8         # only j within max_len of i can yield a dictionary word
9         for j in range(max(0, i - max_len), i):
10             if dp[j] and s[j:i] in words:
11                 dp[i] = True
12                 break
13     return dp[n]

```

**** Error-prone spots:****

- › $dp[0] = \text{True}$ is the essential base case — without it nothing can start.
- › Convert `wordDict` to a set once; repeated in `list` checks make it needlessly $O(n^2 * \text{dict})$.
- › The slice $s[j:i]$ is the candidate word; mixing the bounds ($s[i:j]$) yields empty/wrong

strings.

- › Pruning the inner loop to `range(i - max_len, i)` is a sound optimization (longer slices can't be words) but be careful to clamp the lower bound at 0.
- › Once `dp[i]` is set `True`, `break` — no need to test further split points.

Follow-up. Word Break II (LC 140) asks for *all* segmentations; use DFS with memoization on suffixes, since the count can be exponential and pure DP only answers the boolean.

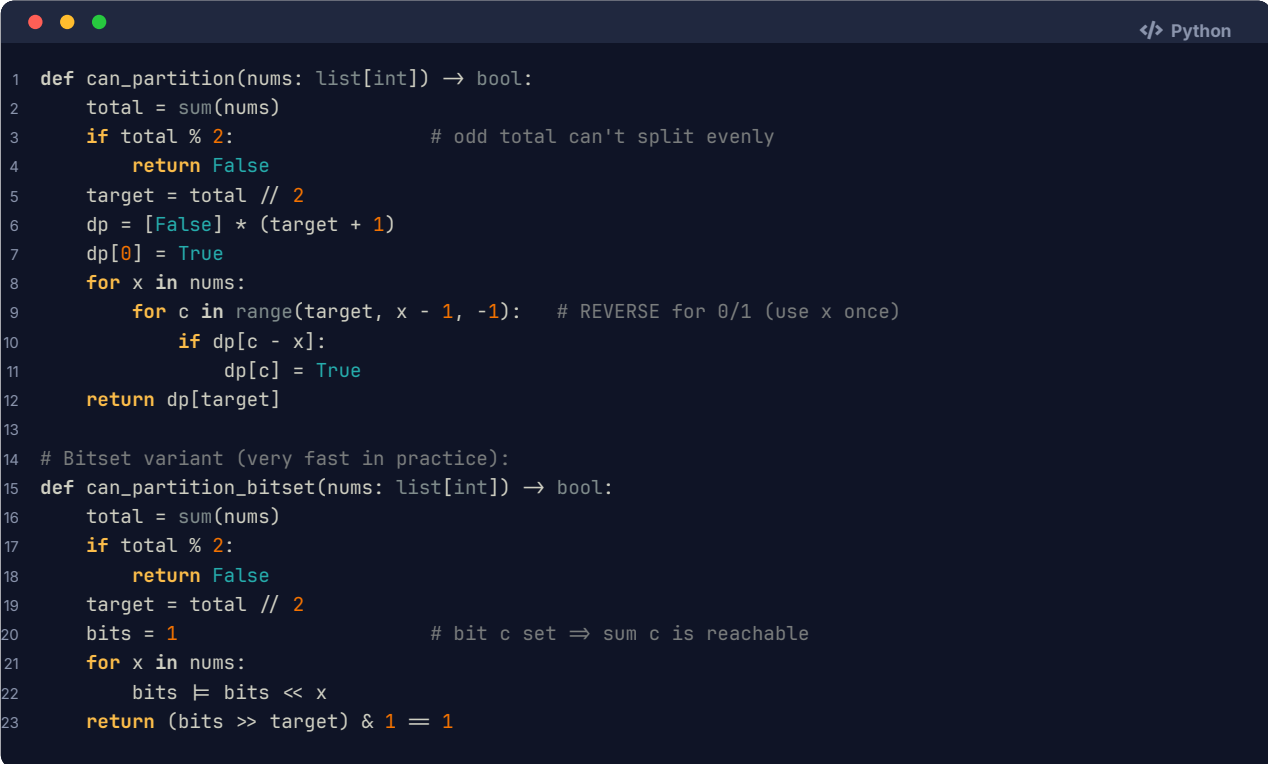
6.17 Partition Equal Subset Sum (LC 416)

Problem. Given an array `nums` of positive integers, determine whether it can be split into two subsets with equal sums.

Optimal — Subset-sum DP · Time $O(n \cdot \text{target})$, Space $O(\text{target})$. `target = total / 2`. A boolean 1D rolling array (effectively a bitset) suffices.

State & recurrence. If the total sum is odd, return `False` immediately. Let `target = total // 2`. Let `dp[c]` = is some subset summable to exactly `c`. Transition (0/1 knapsack on reachability): for each number `x`, `dp[c] = dp[c] or dp[c - x]` for `c` from `target` down to `x`. Base case: `dp[0] = True` (empty subset). Answer: `dp[target]`.

Idea. Reaching exactly half the total with some subset leaves the complement also equal to half — so the question reduces to a subset-sum feasibility test. Each `x` either joins the subset (consuming `x` from the target) or not; the reverse capacity loop enforces 0/1 (each number used at most once).



```

1 def can_partition(nums: list[int]) → bool:
2     total = sum(nums)
3     if total % 2:
4         # odd total can't split evenly
5         return False
6     target = total // 2
7     dp = [False] * (target + 1)
8     dp[0] = True
9     for x in nums:
10        for c in range(target, x - 1, -1): # REVERSE for 0/1 (use x once)
11            if dp[c - x]:
12                dp[c] = True
13    return dp[target]
14
15 # Bitset variant (very fast in practice):
16 def can_partition_bitset(nums: list[int]) → bool:
17     total = sum(nums)
18     if total % 2:
19         return False
20     target = total // 2
21     bits = 1
22     # bit c set ⇒ sum c is reachable
23     for x in nums:
24         bits |= bits << x
25    return (bits >> target) & 1 == 1

```

**** Error-prone spots:****

- › Check `total % 2` first — an odd total is immediately impossible; skipping this wastes work and can give a wrong cell read.
- › **Iterate capacity in REVERSE** (`range(target, x-1, -1)`), exactly as in 0/1 knapsack — each number may be used at most once. A forward loop would allow reusing `x` (unbounded) and

falsely report True.

- › `dp[0] = True` is the base case (empty subset sums to 0).
- › The lower loop bound `x - 1` ensures the smallest processed `c` is `x`; for `c < x` the element can't fit.
- › In the bitset version, `bits` starts at 1 (bit 0 set), and `bits |= bits << x` adds `x` to every currently reachable sum in one shift — a $O(n \cdot \text{target} / \text{word})$ speedup.
- › Numbers must be positive (LC guarantees this); zeros or negatives would need a different formulation.

7 Backtracking

7.1 N-Queens (LC 51)

Problem. Place n queens on an $n \times n$ chessboard so that no two attack each other (no shared row, column, or diagonal). Return all distinct board configurations, each as a list of strings using 'Q' and '.'.

Approach — DFS row-by-row with column/diagonal sets · Time $O(n!)$, Space $O(n)$. Search space: one queen per row, choosing a column for each — at most $n \cdot (n-1) \cdot \dots = O(n!)$ leaf paths. Pruning: a column c on row r is invalid if c is taken, or either diagonal is taken. Track three sets so validity is $O(1)$.

Idea. Each row gets exactly one queen, so we only ever recurse over column choices for the current row. Two cells share a / diagonal iff $r + c$ is constant, and a \ diagonal iff $r - c$ is constant. Maintaining `cols`, `diag (r-c)`, and `anti (r+c)` as sets lets us test and place a queen in $O(1)$ instead of scanning the board. Undo the three set insertions when we backtrack.

```

1 from typing import List
2
3 def solveNQueens(n: int) -> List[List[str]]:
4     res = []
5     cols, diag, anti = set(), set(), set() # diag = r-c, anti = r+c
6     placement = [-1] * n # placement[r] = column of queen in row r
7
8     def backtrack(r: int) -> None:
9         if r == n:
10             board = [ "." * c + "Q" + "." * (n - c - 1) for c in placement ]
11             res.append(board)
12             return
13         for c in range(n):
14             if c in cols or (r - c) in diag or (r + c) in anti:
15                 continue
16             cols.add(c); diag.add(r - c); anti.add(r + c)
17             placement[r] = c
18             backtrack(r + 1)
19             cols.remove(c); diag.remove(r - c); anti.remove(r + c) # undo
20
21     backtrack(0)
22     return res

```

**** Error-prone spots:****

- › Diagonal keys: / uses $r + c$ and \ uses $r - c$. Swapping them silently still "works" for small n but is wrong; keep them distinct sets.

- › $r - c$ can be negative — fine for a set, but if you switch to a boolean array you must offset by $n - 1$.
- › Forgetting to remove all three set entries on backtrack corrupts later branches.
- › Building the board string: `"." * c + "Q" + "." * (n - c - 1)` — off-by-one in the trailing dots is common.
- › Do not reset `placement[r]` between sibling choices unless you read it only for rows $< r$; here we always overwrite before recursing, so it is safe.

Follow-up. For only the *count* (LC 52), drop `res/placement` and return an integer; the bitmask variant uses ints for `cols/diag/anti` and `available = ~(cols|diag|anti) & ((1<<n)-1)`, iterating set bits via `p = available & -available`.

7.2 Sudoku Solver (LC 37)

Problem. Fill a partially completed 9×9 Sudoku grid (in place) so every row, column, and 3×3 box contains digits 1–9 exactly once. Empty cells are `'.'`. A unique solution is guaranteed.

Approach — Backtracking with constraint sets · Time $O(9^{(\text{empty cells})})$ worst case, Space $O(1)$ extra (81 fixed slots). Search space: assign a digit to each empty cell. Pruning (constraint propagation): for a cell, only try digits not already present in its row, column, or box; precomputed sets make each check $O(1)$.

Idea. Maintain `rows[i]`, `cols[j]`, and `boxes[(i//3)*3 + j//3]` as sets of used digits. We pick the *next* empty cell, try each legal digit, recurse, and undo on failure. Choosing the most-constrained cell first (fewest candidates) drastically prunes the tree, but a simple left-to-right scan is already fast given the guaranteed unique solution.

```

1 from typing import List
2
3 def solveSudoku(board: List[List[str]]) → None:
4     rows = [set() for _ in range(9)]
5     cols = [set() for _ in range(9)]
6     boxes = [set() for _ in range(9)]
7     empties = []
8
9     def box_id(r: int, c: int) → int:
10         return (r // 3) * 3 + c // 3
11
12     for r in range(9):
13         for c in range(9):
14             ch = board[r][c]
15             if ch == '.':
16                 empties.append((r, c))
17             else:
18                 rows[r].add(ch); cols[c].add(ch); boxes[box_id(r, c)].add(ch)
19
20     def backtrack(k: int) → bool:
21         if k == len(empties):
22             return True
23         r, c = empties[k]
24         b = box_id(r, c)
25         for d in "123456789":
26             if d in rows[r] or d in cols[c] or d in boxes[b]:
27                 continue
28             board[r][c] = d
29             rows[r].add(d); cols[c].add(d); boxes[b].add(d)

```

```

30         if backtrack(k + 1):
31             return True
32         board[r][c] = '.' # undo
33         rows[r].remove(d); cols[c].remove(d); boxes[b].remove(d)
34     return False
35
36     backtrack(0)

```

**** Error-prone spots:****

- › Box index must be $(r // 3) * 3 + c // 3$; a common bug is $r // 3 + c // 3$ which collapses distinct boxes.
- › Return True/False to *stop* once solved — without short-circuiting you keep searching and may overwrite a valid solution.
- › Undo all three sets *and* the board cell; missing the board reset leaves stale digits.
- › Use the digit characters ('1' .. '9') to match the input type, not ints.
- › Modify board in place; returning a new board fails the LeetCode signature.

Follow-up. Most-constrained-variable heuristic: pick the empty cell with the fewest legal digits each step. Bitmask rows/cols/boxes as 9-bit ints to shrink memory and speed candidate enumeration.

7.3 Generate Parentheses (LC 22)

Problem. Given n pairs of parentheses, generate all combinations of well-formed (balanced) parentheses strings.

Approach — Backtracking with open/close counters · Time $O(4^n / \sqrt{n})$, Space $O(n)$ recursion. Search space: binary tree of "add (" vs "add)" of depth $2n$. Pruning: add (only while $\text{open} < n$; add) only while $\text{close} < \text{open}$ (so we never produce an unmatched). The number of valid strings is the n -th Catalan number, $\sim 4^n / (n^{1.5} * \sqrt{\pi})$.

Idea. A prefix is extendable to a valid string iff at every point the count of) never exceeds (and total (never exceeds n . By enforcing these two invariants as we build, every leaf at length $2n$ is automatically valid — no post-hoc validation needed.

```

1  from typing import List
2
3  def generateParenthesis(n: int) -> List[str]:
4      res = []
5      path = []
6
7      def backtrack(open_ct: int, close_ct: int) -> None:
8          if len(path) == 2 * n:
9              res.append("".join(path))
10             return
11         if open_ct < n:
12             path.append("(")
13             backtrack(open_ct + 1, close_ct)
14             path.pop() # undo
15         if close_ct < open_ct:
16             path.append(")")
17             backtrack(open_ct, close_ct + 1)

```

```

18         path.pop()                # undo
19
20     backtrack(0, 0)
21     return res

```

**** Error-prone spots:****

- › The close condition is `close_ct < open_ct`, NOT `close_ct < n`; the latter generates invalid strings like `()()`.
- › Pop after each recursive call to keep path consistent for the sibling branch.
- › Termination is on total length $2n$, not on `open_ct == n` alone (you would miss the trailing `)`s).
- › If you pass strings by value instead of using a shared path, you do not need to undo — but the list-with-pop version is more efficient and is the idiomatic backtracking form.

Follow-up. Closed-form count is the Catalan number $c(2n, n)/(n+1)$. The same open/close invariant powers validity checks and the “remove invalid parentheses” problem.

7.4 Subsets (LC 78)

Problem. Given an array `nums` of *distinct* integers, return all 2^n subsets (the power set). The result must not contain duplicate subsets; order does not matter.

Approach — Backtracking, include/exclude per element · Time $O(n * 2^n)$, Space $O(n)$ recursion. Search space: for each index decide take-or-skip, a binary tree with 2^n leaves. No pruning needed (every path is a valid subset); the $O(n)$ factor is the cost of copying each subset into the output.

Idea. Standard skeleton: iterate a start index, and for each element from start onward, *choose it*, record the current path as one subset, recurse with the next index, then *unchoose*. Recording the path at *every* node (not just leaves) yields all subsets, because each distinct prefix is itself a subset.

```

1  from typing import List
2
3  def subsets(nums: List[int]) → List[List[int]]:
4      res = []
5      path = []
6
7      def backtrack(start: int) → None:
8          res.append(path[:])          # record every node, copy!
9          for i in range(start, len(nums)):
10             path.append(nums[i])      # choose
11             backtrack(i + 1)          # recurse with i+1, not start+1
12             path.pop()                # unchoose
13
14     backtrack(0)
15     return res

```

**** Error-prone spots:****

- › Append a *copy* `path[:]`; appending `path` stores a reference that later mutations clobber to `[]`.
- › Recurse with `i + 1` (not `start + 1`) so each element is considered once and subsets stay in non-decreasing index order — preventing permutations/duplicates.

- › Record the subset *before* the loop (at entry), so the empty subset and all intermediate prefixes are captured.
- › For LC 90 (Subsets II, with duplicates): sort first, then skip duplicates with `if i > start and nums[i] == nums[i-1]: continue`.

Follow-up. Bitmask enumeration: for `mask` in `range(1 << n)`, include `nums[i]` where `mask >> i & 1`. Clean and iterative, same $O(n * 2^n)$ cost.

7.5 Permutations (LC 46)

Problem. Given an array `nums` of *distinct* integers, return all $n!$ permutations.

Approach — Backtracking with a used set / in-place swap · Time $O(n * n!)$, Space $O(n)$.

Search space: at depth d we choose among the $n - d$ unused elements, giving $n!$ leaves. Pruning: skip already-used elements via a boolean `used` array (or swap unused ones to the front). The $O(n)$ factor copies each full permutation out.

Idea. Unlike subsets, position matters and we reuse no start index — every unused element is a candidate at every depth. A `used` array marks elements currently on the path; mark before recursing and unmark on the way back. When `len(path) == n`, we have a complete permutation.



```

1 from typing import List
2
3 def permute(nums: List[int]) -> List[List[int]]:
4     res = []
5     path = []
6     used = [False] * len(nums)
7
8     def backtrack() -> None:
9         if len(path) == len(nums):
10             res.append(path[:])          # copy
11             return
12         for i in range(len(nums)):
13             if used[i]:
14                 continue
15             used[i] = True
16             path.append(nums[i])
17             backtrack()
18             path.pop()                  # undo
19             used[i] = False             # undo
20
21     backtrack()
22     return res

```

**** Error-prone spots:****

- › No start index here — looping from 0 every call is what distinguishes permutations from combinations/subsets.
- › Undo *both* `path.pop()` and `used[i] = False`; forgetting the latter loses permutations.
- › Copy `path[:]` at the leaf.
- › For LC 47 (Permutations II, with duplicates): sort, then `if used[i] or (i > 0 and nums[i] == nums[i-1] and not used[i-1]): continue` — the `not used[i-1]` enforces a canonical order among equal elements so each distinct permutation is produced once.

Follow-up. The in-place swap variant avoids the `used` array: swap `nums[start]` with each `nums[i]`

$\geq \text{start}]$, recurse on $\text{start}+1$, swap back. It generates permutations in a different order and uses $O(1)$ extra besides recursion.

7.6 Combinations (LC 77)

Problem. Return all combinations of k numbers chosen from the range $[1, n]$ (order within a combination does not matter).

Approach — Backtracking with start index + length cap · Time $O(k * C(n, k))$, Space $O(k)$.
Search space: choose k increasing values out of n , $C(n, k)$ leaves. Pruning: stop recursing once enough elements remain — if fewer than $k - \text{len}(\text{path})$ candidates are left, abandon the branch.

Idea. Same skeleton as subsets but we only record paths of exactly length k . The start index enforces increasing selection so $\{1, 2\}$ and $\{2, 1\}$ are not both produced. A strong pruning bound: the last useful starting value is $n - (k - \text{len}(\text{path})) + 1$, because we need $k - \text{len}(\text{path})$ more elements.



```

1 from typing import List
2
3 def combine(n: int, k: int) -> List[List[int]]:
4     res = []
5     path = []
6
7     def backtrack(start: int) -> None:
8         if len(path) == k:
9             res.append(path[:])          # copy
10            return
11        need = k - len(path)
12        # i can go at most up to n - need + 1 so enough elements remain
13        for i in range(start, n - need + 2):
14            path.append(i)
15            backtrack(i + 1)              # next picks are strictly larger
16            path.pop()                    # undo
17        # note loop range is in [start, n - need + 1] inclusive
18
19    backtrack(1)
20    return res

```

**** Error-prone spots:****

- Numbers are $1..n$ inclusive, so start at 1 and the upper bound in range must reach n (use $n + 1$ without pruning, $n - \text{need} + 2$ with pruning).
- Pruning bound: $n - \text{need} + 2$ as the exclusive range end equals the inclusive last value $n - \text{need} + 1$; off-by-one here either misses combinations or wastes work.
- Recurse with $i + 1$ to keep values strictly increasing (no repeats, no permutations).
- Record only when $\text{len}(\text{path}) = k$, unlike subsets which records at every node.
- Copy the path at the leaf.

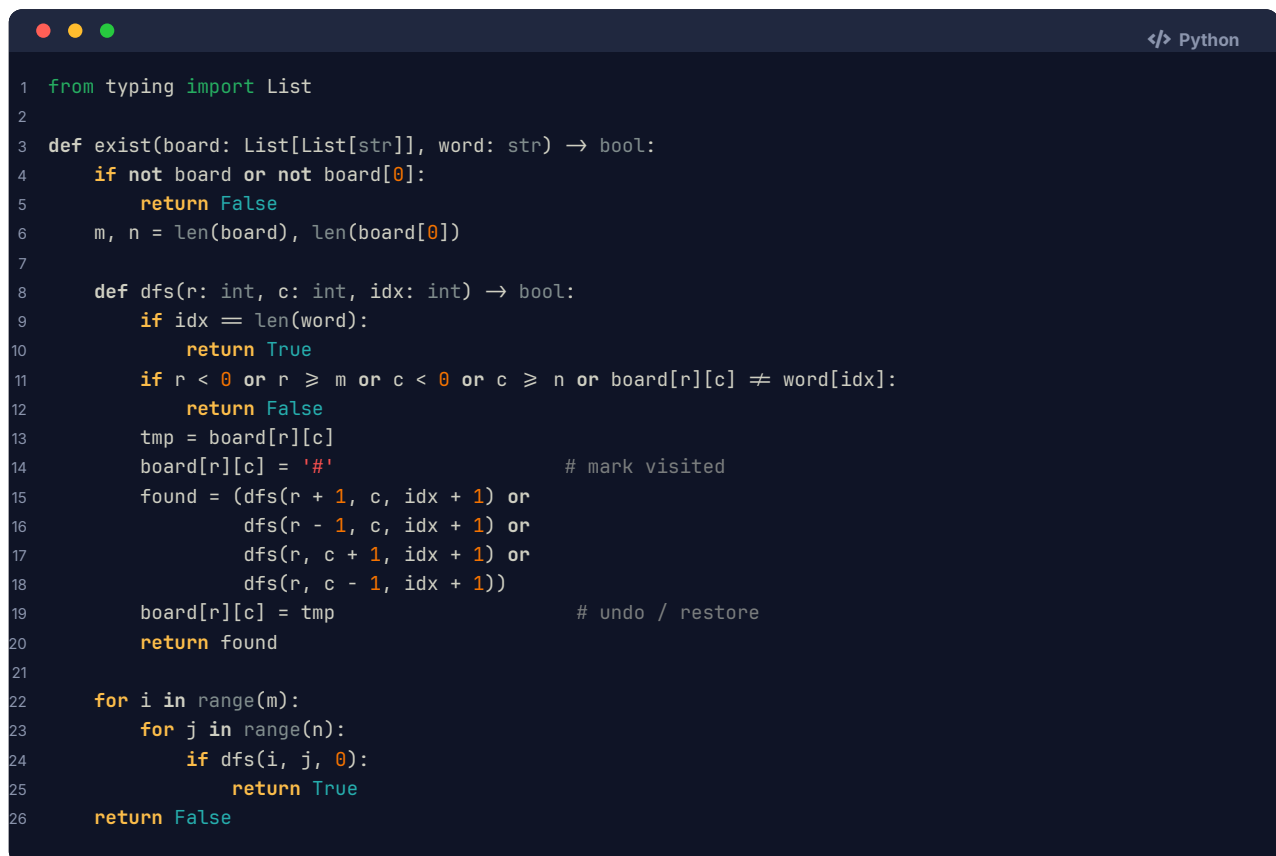
Follow-up. Shared skeleton: Subsets (record every node, $i+1$), Combinations (record at depth k , $i+1$), Permutations (no start, used array). For Combination Sum (LC 39, reuse allowed) recurse with i instead of $i + 1$.

7.7 Word Search (LC 79)

Problem. Given an $m \times n$ board of characters and a string word, return True if word can be constructed from sequentially adjacent (horizontally/vertically) cells, where the same cell may not be used more than once.

Approach — DFS from each cell with in-place visited marking · Time $O(m * n * 4^L)$, Space $O(L)$ recursion ($L = \text{len}(\text{word})$). Search space: start DFS at every cell, branching to up to 4 neighbors at each of L depths. Pruning: stop immediately when the current cell does not match $\text{word}[\text{idx}]$, or is out of bounds, or is already on the path.

Idea. From a matching start cell we explore the 4 neighbors, advancing idx . To avoid revisiting a cell within the same path we mark it (temporarily overwrite with a sentinel like '#'), recurse, then restore it — this is the backtracking undo. As soon as $\text{idx} = \text{len}(\text{word})$ we have matched the whole word.



```

1  from typing import List
2
3  def exist(board: List[List[str]], word: str) → bool:
4      if not board or not board[0]:
5          return False
6      m, n = len(board), len(board[0])
7
8      def dfs(r: int, c: int, idx: int) → bool:
9          if idx == len(word):
10             return True
11             if r < 0 or r ≥ m or c < 0 or c ≥ n or board[r][c] ≠ word[idx]:
12                 return False
13             tmp = board[r][c]
14             board[r][c] = '#'          # mark visited
15             found = (dfs(r + 1, c, idx + 1) or
16                     dfs(r - 1, c, idx + 1) or
17                     dfs(r, c + 1, idx + 1) or
18                     dfs(r, c - 1, idx + 1))
19             board[r][c] = tmp          # undo / restore
20             return found
21
22     for i in range(m):
23         for j in range(n):
24             if dfs(i, j, 0):
25                 return True
26     return False

```

**** Error-prone spots:****

- Check $\text{idx} = \text{len}(\text{word})$ *before* the bounds/match check, so a fully matched word returns True even if it ends at the board edge.
- Restore $\text{board}[\text{r}][\text{c}] = \text{tmp}$ on *every* return path; a sentinel left behind blocks valid future paths.
- The sentinel ('#') must not appear in word; restoring the original char makes this robust regardless.
- Short-circuit with `or` so the first successful direction stops further search.
- Start DFS from *every* cell — the word may begin anywhere.

Follow-up. For Word Search II (LC 212, many words) build a Trie of the words and DFS the board once against the Trie, pruning whole subtrees; prune Trie leaves after a word is found to avoid duplicates.

8 Greedy

8.1 Merge Intervals (LC 56)

Problem. Given an array of intervals $[start, end]$, merge all overlapping intervals and return the non-overlapping intervals that cover the same ranges.

Approach — Sort by start, sweep and merge · Time $O(n \log n)$, Space $O(n)$ output ($O(1)$ extra).

Greedy choice: process intervals in increasing start order; the current "open" merged interval absorbs the next interval iff they overlap. *Why safe (exchange argument):* after sorting by start, any interval overlapping the merged block must overlap the *current* one, since all later starts are $\geq$ the current start; thus a single left-to-right pass never misses or wrongly merges.

Idea. Sort by start. Keep the last merged interval; for each next interval, if its start $\leq$ the last merged end, extend the end to $\max(end, next.end)$; otherwise the gap is real, so push it as a new block.

```

1 from typing import List
2
3 def merge(intervals: List[List[int]]) -> List[List[int]]:
4     intervals.sort(key=lambda iv: iv[0])
5     merged = []
6     for s, e in intervals:
7         if merged and s <= merged[-1][1]: # overlap (touching counts)
8             merged[-1][1] = max(merged[-1][1], e)
9         else:
10            merged.append([s, e])
11    return merged

```

**** Error-prone spots:****

- › Sort by *start*. Sorting by end (correct for other problems) breaks the merge invariant here.
- › Use $\max(merged[-1][1], e)$ — the next interval may be fully contained (e smaller), so blindly assigning $merged[-1][1] = e$ shrinks the block.
- › Overlap test: $s \leq merged[-1][1]$ treats touching intervals like $[1, 2], [2, 3]$ as overlapping (merge to $[1, 3]$). If touching should *not* merge, use strict $<$.
- › Append a fresh list $[s, e]$, not a reference to the input row, if you later mutate it.

Follow-up. Insert Interval (LC 57): given an already-sorted list, splice the new interval in $O(n)$ without re-sorting by collecting before/overlap/after segments.

8.2 Non-overlapping Intervals (LC 435)

Problem. Given a set of intervals, return the minimum number of intervals to remove so the rest are non-overlapping.

Approach — Sort by end, greedily keep earliest-finishing · Time $O(n \log n)$, Space $O(1)$. This is the classic activity-selection problem. Greedy choice: always keep the interval that *ends earliest* among remaining compatible ones. *Why safe (exchange argument):* if an optimal solution keeps an interval ending later than the earliest-finishing compatible one, swapping in the earliest-finisher leaves at least as much room for the rest, so it is also optimal. Maximizing kept intervals minimizes removals.

Idea. Sort by end. Track `prev_end`; scan intervals — if the current start $\geq$ `prev_end`, it does not

overlap the last kept interval, so keep it and advance `prev_end`. Otherwise it overlaps and must be removed (count it).

```

1 from typing import List
2
3 def eraseOverlapIntervals(intervals: List[List[int]]) → int:
4     if not intervals:
5         return 0
6     intervals.sort(key=lambda iv: iv[1])    # sort by END
7     removals = 0
8     prev_end = float('-inf')
9     for s, e in intervals:
10        if s ≥ prev_end:                # non-overlapping → keep
11            prev_end = e
12        else:                          # overlaps → remove this one
13            removals += 1
14    return removals

```

**** Error-prone spots:****

- › Sort by *end*, not start; sorting by start gives a wrong greedy here.
- › Comparison is $s \geq \text{prev_end}$ (non-strict): intervals like $[1, 2], [2, 3]$ only touch and are *not* considered overlapping per LeetCode's definition. If shared endpoints counted as overlap you would use $s > \text{prev_end}$.
- › When overlapping, keep the one already chosen (earlier end) and remove the current — do *not* update `prev_end`. Updating it on overlap is the most common bug.
- › Initialize `prev_end` to `-inf` so the first interval is always kept.

Follow-up. Maximum number of non-overlapping intervals kept is $n - \text{removals}$. Same template solves "max meetings in one room" and is the dual of interval scheduling.

8.3 Jump Game (LC 55)

Problem. Given an array `nums` where `nums[i]` is the maximum jump length from index `i`, determine whether you can reach the last index starting from index 0.

Approach — Greedy farthest-reachable scan · Time $O(n)$, Space $O(1)$. Greedy choice: track the farthest index reachable so far; extend it as you sweep. *Why safe:* reachability is monotone — if index `i` is reachable and $i + \text{nums}[i] \geq j$, then `j` is reachable; the single best frontier farthest dominates any individual interval, so we never need to revisit earlier positions.

Idea. Maintain `farthest`, the maximum index reachable using positions seen so far. Iterate `i` from 0; if $i > \text{farthest}$, there is a gap we cannot cross — return `False`. Otherwise update `farthest = max(farthest, i + nums[i])`. If `farthest` reaches the last index, succeed.

```

1 from typing import List
2
3 def canJump(nums: List[int]) → bool:
4     farthest = 0
5     last = len(nums) - 1
6     for i, step in enumerate(nums):
7         if i > farthest:                # current index unreachable
8             return False

```



```

9         farthest = max(farthest, i + step)
10        if farthest ≥ last:
11            return True
12    return True

```

**** Error-prone spots:****

- › Check $i > \text{farthest}$ *before* using `nums[i]`; once you fall behind the frontier, every later index is also unreachable.
- › A zero at index i is fine as long as $\text{farthest} > i$ from an earlier jump; only a zero that the frontier cannot pass is fatal.
- › The early `return True` when $\text{farthest} \geq \text{last}$ is an optimization; the final `return True` covers single-element arrays ($\text{last} = 0$, loop reaches it).
- › Do not confuse `farthest` (an index) with a remaining-fuel counter; the index form generalizes cleanly to Jump Game II.

Follow-up. Backward greedy: track the leftmost good index initialized to `last`; if $i + \text{nums}[i] \geq \text{good}$ then $\text{good} = i$. Reachable iff $\text{good} = 0$ at the end.

8.4 Jump Game II (LC 45)

Problem. Same setup as LC 55, but it is guaranteed you can reach the last index. Return the *minimum* number of jumps to get from index 0 to the last index.

Approach — Greedy BFS by levels · Time $O(n)$, Space $O(1)$. Think of indices reachable in k jumps as BFS level k . Greedy choice: from the current level $[\text{start}, \text{cur_end}]$, the next level extends to the farthest index reachable from any position in it; we commit one jump per level boundary. *Why safe:* every index in the current level needs exactly the same jump count; the farthest reach over the whole level is the optimal frontier for the next jump, so taking it never overshoots the minimum.

Idea. Sweep i while maintaining `cur_end` (the boundary of the current jump's reach) and `farthest` (the best reach seen within the current window). When i hits `cur_end`, we must take a jump: increment `jumps` and set `cur_end = farthest`. We stop the loop at $\text{len} - 1$ so we do not count an extra jump after arriving.

```

1  from typing import List
2
3  def jump(nums: List[int]) → int:
4      jumps = 0
5      cur_end = 0
6      farthest = 0
7      for i in range(len(nums) - 1):          # stop before the last index
8          farthest = max(farthest, i + nums[i])
9          if i == cur_end:                    # exhausted current jump's range
10             jumps += 1
11             cur_end = farthest
12    return jumps

```

**** Error-prone spots:****

- › Loop to $\text{len}(\text{nums}) - 1$ (exclusive of the last index). Including the last index can over-count

by one extra jump.

- › Increment jumps exactly when $i = \text{cur_end}$, the level boundary — not on every index.
- › `farthest` must be updated for the current i *before* the boundary check, so the new `cur_end` reflects this position's reach.
- › Initialize all three to 0; for a single-element array the loop body never runs and 0 jumps is correct.
- › This $O(n)$ greedy beats the $O(n^2)$ DP; do not fall back to nested loops.

Follow-up. The level interpretation makes it a BFS in disguise; the same windowed-frontier idea solves "minimum taps to water a garden" (LC 1326) after mapping each tap to its reach interval.

8.5 Task Scheduler (LC 621)

Problem. Given a list of CPU tasks (chars) and a cooldown n , where two identical tasks must be at least n intervals apart, return the *minimum* total intervals (including idle slots) to finish all tasks.

Approach — Greedy by most-frequent task · Time $O(T)$ (or $O(T + 26 \log 26)$), Space $O(1)$ (26 letters). Greedy choice: schedule the most frequent task as early and as often as cooldown allows; it dictates the skeleton of idle slots. *Why safe:* the busiest task forces a frame of $(\text{maxFreq} - 1)$ blocks each of length $(n + 1)$, plus a final group; every other task can only fit into that frame's gaps. The optimum is therefore either that frame length (with idles) or simply $\text{len}(\text{tasks})$ when there are enough distinct tasks to fill all gaps.

Idea. Let `maxFreq` be the highest task count and `count_max` how many tasks share it. The frame is $(\text{maxFreq} - 1) * (n + 1)$ slots, then add `count_max` for the final partial block. If there are so many distinct tasks that idle slots all get filled, the answer is just $\text{len}(\text{tasks})$. Take the max of the two — that handles the "no idle needed" case without an explicit simulation.

```

1 from typing import List
2 from collections import Counter
3
4 def leastInterval(tasks: List[str], n: int) -> int:
5     counts = Counter(tasks)
6     max_freq = max(counts.values())
7     count_max = sum(1 for c in counts.values() if c == max_freq)
8     # frame built around the most frequent task(s)
9     frame = (max_freq - 1) * (n + 1) + count_max
10    return max(frame, len(tasks))

```

**** Error-prone spots:****

- › Block width is $n + 1$ (one slot for the task itself plus n cooldown slots), not n .
- › Use $(\text{max_freq} - 1)$ blocks, not `max_freq`; the last occurrence forms the trailing group counted by `count_max`.
- › Add `count_max` (number of tasks tied for the maximum), not 1 — multiple tasks can finish in the final block.
- › The `max(frame, len(tasks))` guard is essential: when distinct tasks fill every gap, no idling occurs and the answer is $\text{len}(\text{tasks})$ (the formula could otherwise underestimate).
- › n can be 0 (no cooldown), giving $\max(\text{len}(\text{tasks}), \text{len}(\text{tasks})) = \text{len}(\text{tasks})$ — the formula handles it.

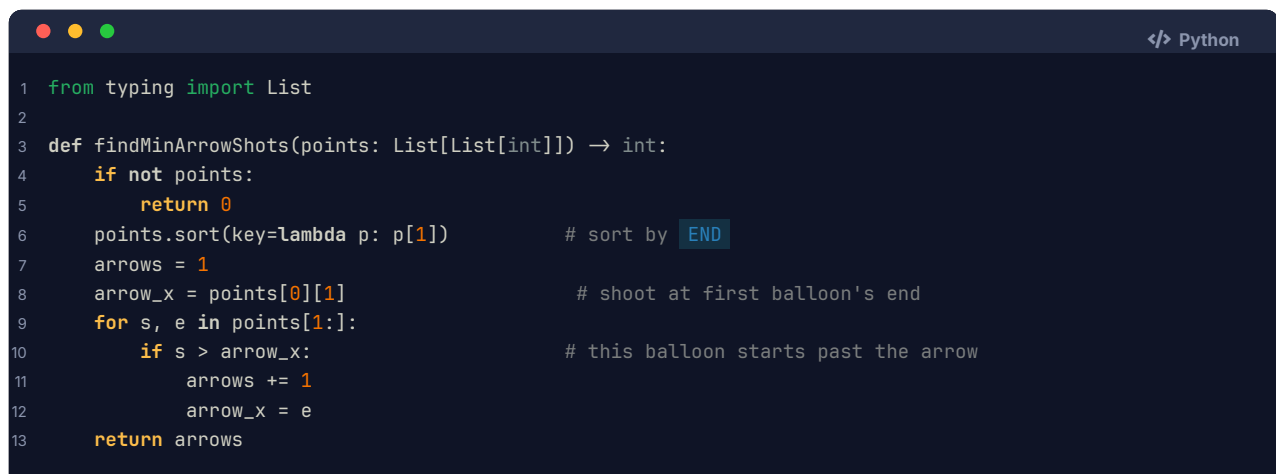
Follow-up. To output an actual schedule (not just the count), use a max-heap of remaining counts plus a cooldown queue holding (ready_time, count); pop the most frequent ready task each tick. That is $O(T \log 26)$.

8.6 Minimum Number of Arrows to Burst Balloons (LC 452)

Problem. Each balloon is a horizontal interval $[x_start, x_end]$. An arrow shot vertically at x bursts every balloon with $x_start \leq x \leq x_end$. Return the minimum arrows needed to burst all balloons.

Approach — Sort by end, greedy shoot at current end · Time $O(n \log n)$, Space $O(1)$. Greedy choice: sort by right endpoint; shoot an arrow at the end of the earliest-finishing balloon, which bursts all balloons overlapping that point. *Why safe (exchange argument):* placing the arrow at the smallest end among an overlapping group bursts that balloon and as many others as possible; any optimal solution can be shifted to shoot at this point without using more arrows, so greedy is optimal. This is interval-scheduling/stabbing in disguise.

Idea. Sort balloons by end. Fire the first arrow at the first balloon's end. For each subsequent balloon, if its start is $\leq$ the current arrow position, it is already burst — skip it. Otherwise it starts after the arrow, so we need a new arrow placed at this balloon's end.



```

1 from typing import List
2
3 def findMinArrowShots(points: List[List[int]]) → int:
4     if not points:
5         return 0
6     points.sort(key=lambda p: p[1])          # sort by END
7     arrows = 1
8     arrow_x = points[0][1]                  # shoot at first balloon's end
9     for s, e in points[1:]:
10        if s > arrow_x:                       # this balloon starts past the arrow
11            arrows += 1
12            arrow_x = e
13    return arrows

```

**** Error-prone spots:****

- › Sort by *end*; sorting by start gives a wrong count.
- › Overlap test is $s > \text{arrow_x}$ (strict). Touching balloons like $[1,2], [2,3]$ *share* $x = 2$, so one arrow bursts both — a non-strict $\geq$ would over-count arrows.
- › Handle the empty input (return 0) before indexing $\text{points}[0]$.
- › Coordinates can be large/negative; comparing the raw integers is fine, but if you sum or offset them watch for overflow in other languages (not Python).
- › This is the complement of LC 435: there you *remove* overlaps; here you *stab* overlapping groups. Both sort by end.

Follow-up. Equivalent to the minimum number of points that “stab” all intervals — a staple of interval greedy; the same arrow-at-earliest-end logic computes the maximum set of pairwise-overlapping intervals if you instead count the largest stabbed group.